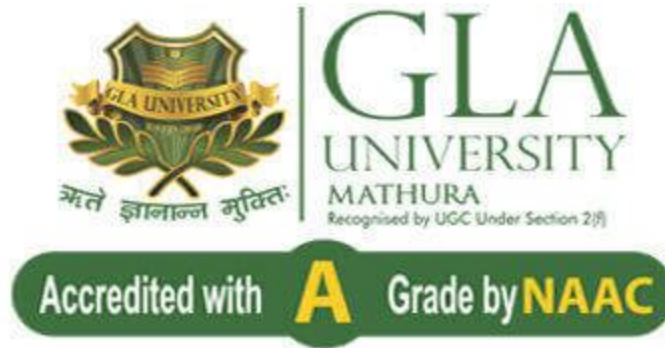




COMPUTER SYSTEM ARCHITECTURE (BCAC0025)

Course: BCA II Year IV Semester

Mr. Aman Deep Singh

Assistant Professor

Instruction Code

- **Program**

- A program is a set of instructions that specify the operation operands and the sequence by which processing has to occur.

- **Computer Instruction**

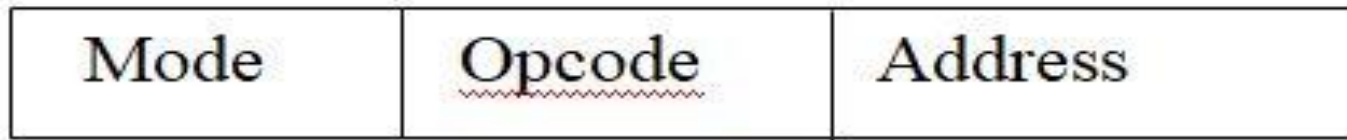
- A computer instruction is a binary code that specifies a sequence of microoperations for the computer.
- The computer reads the each instruction from memory and places it in a control register.
- The control register interpret the binary code of instruction and proceeds to execute it by issuing a sequence of microoperation.

Instruction Code

- An instruction code is a group of bits that instruct the computer to perform a specific operation.

Example : ADD457

- Instruction is usually divided into three parts, each having interpretation.

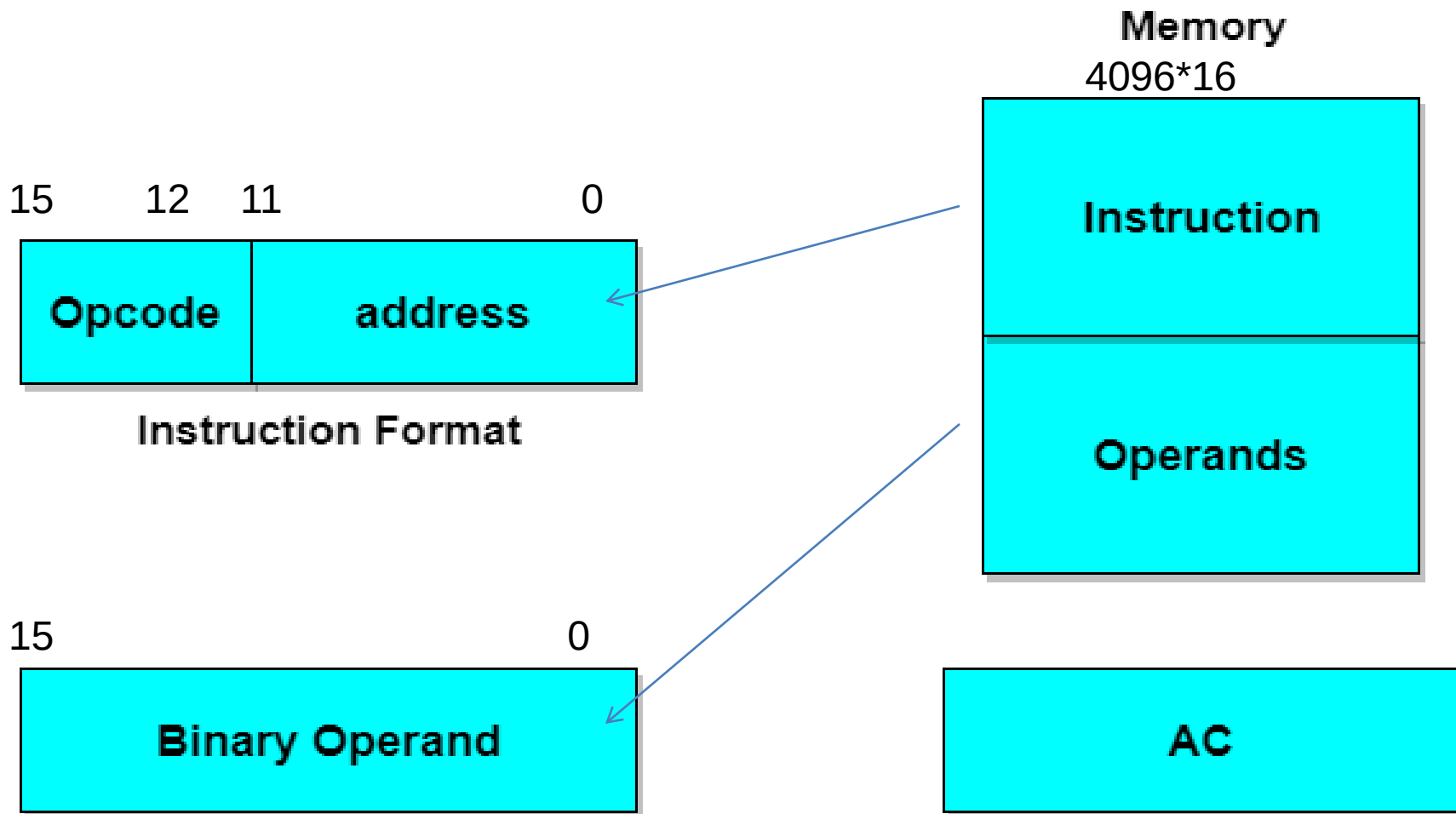


Instruction Format

- **Operation code(Opcod)**
- Operation code of an instruction is a group of bits that define such operation as add, subtract ,multiply , shift and complement.
- The number of bits required for the operation code of an instruction depends on the total number of operations available in computer
- the operation code must contains n bit for given 2^n (or less) different operation.

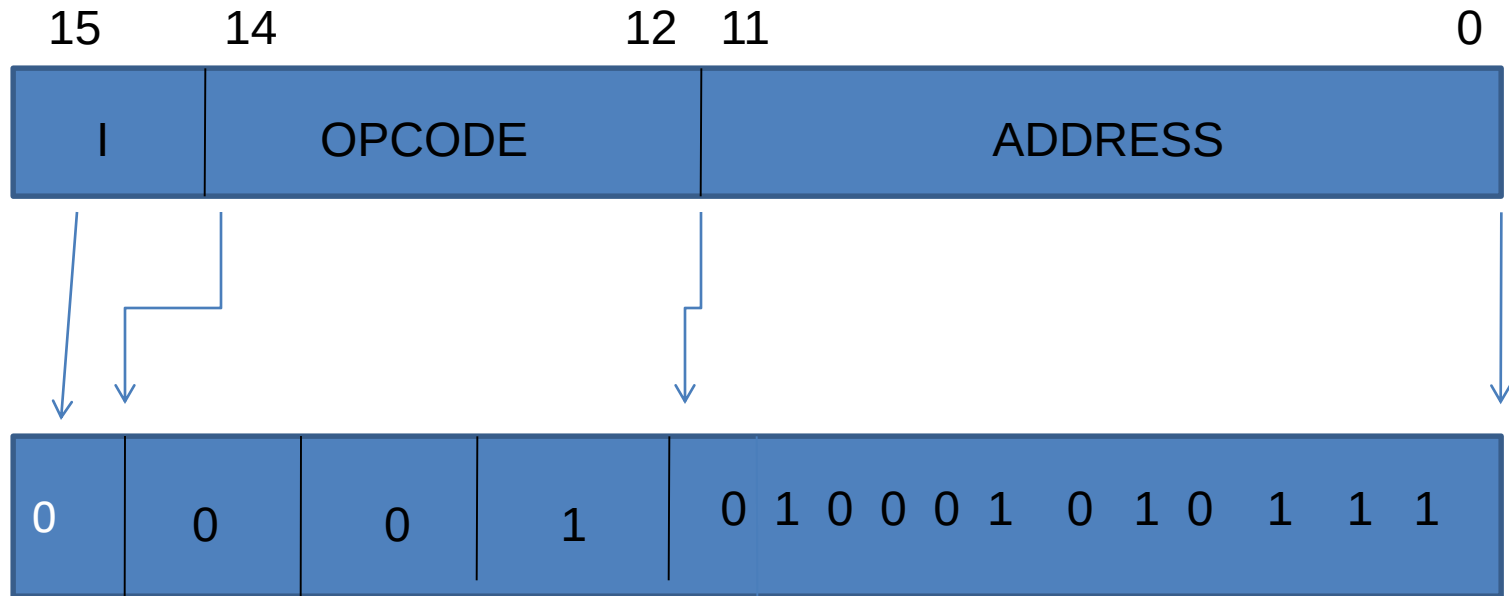
- **For example :**
- Consider a computer with 64 distinct operations, one of them being an ADD operation, the Opcode consist of 6 bits with a bit configuration are grant to ADD operation is 110010.
- “An operation code is some time called a Macro-operation because it specifies a set of micro operations.”

- **Address**
- This part of the instruction specifies the address of the operand.
- **Example:**
- Instruction are stored in one section of memory and data in another. for a memory unit with 4096 words. there are 12 bits are required to specify address($2^{12}=4096$).if we store each instruction code in one 16 bit memory word then 4 bits are required for the operation code.



Instruction format of basic computer

- Instruction format



Add Instruction –ADD457

Address is of two types:

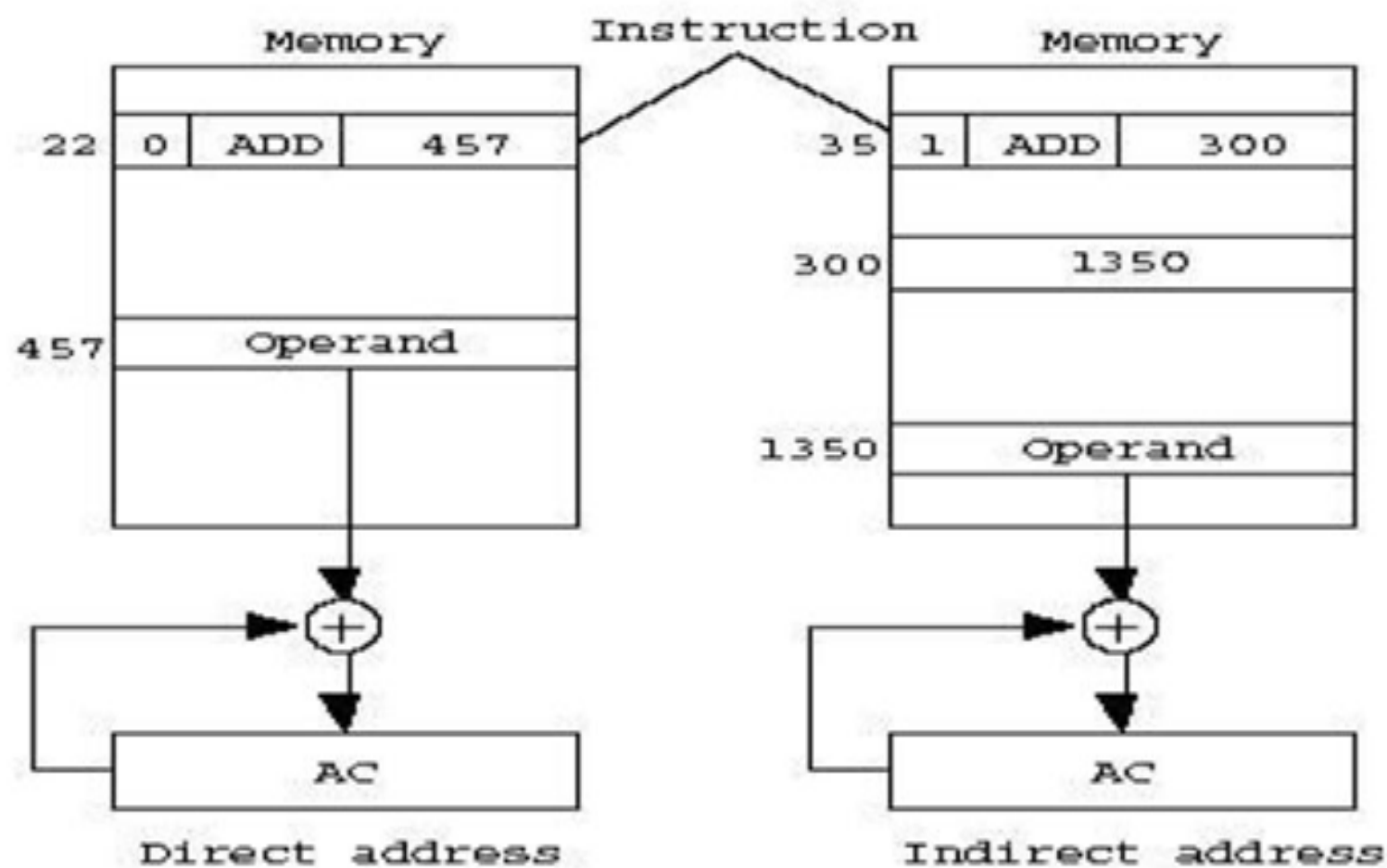
Direct address

- When the second part of instruction code specifies the address of an operand, instruction is said to have direct address.
- it can be identified with the value of I. if $I=0$ then address is direct address.

- **Indirect address**
- When the second part of the instruction code specifies the address of memory word in which address of operand are specified.
- it can be identified with the value of I. if $I=1$ then address is indirect address.
- **Effective address:-**
- It is the address of operand where operand has been stored. In direct address ,the effective address is 457 and in indirect address, effective address is 1350.



Instruction format



Timing and control :

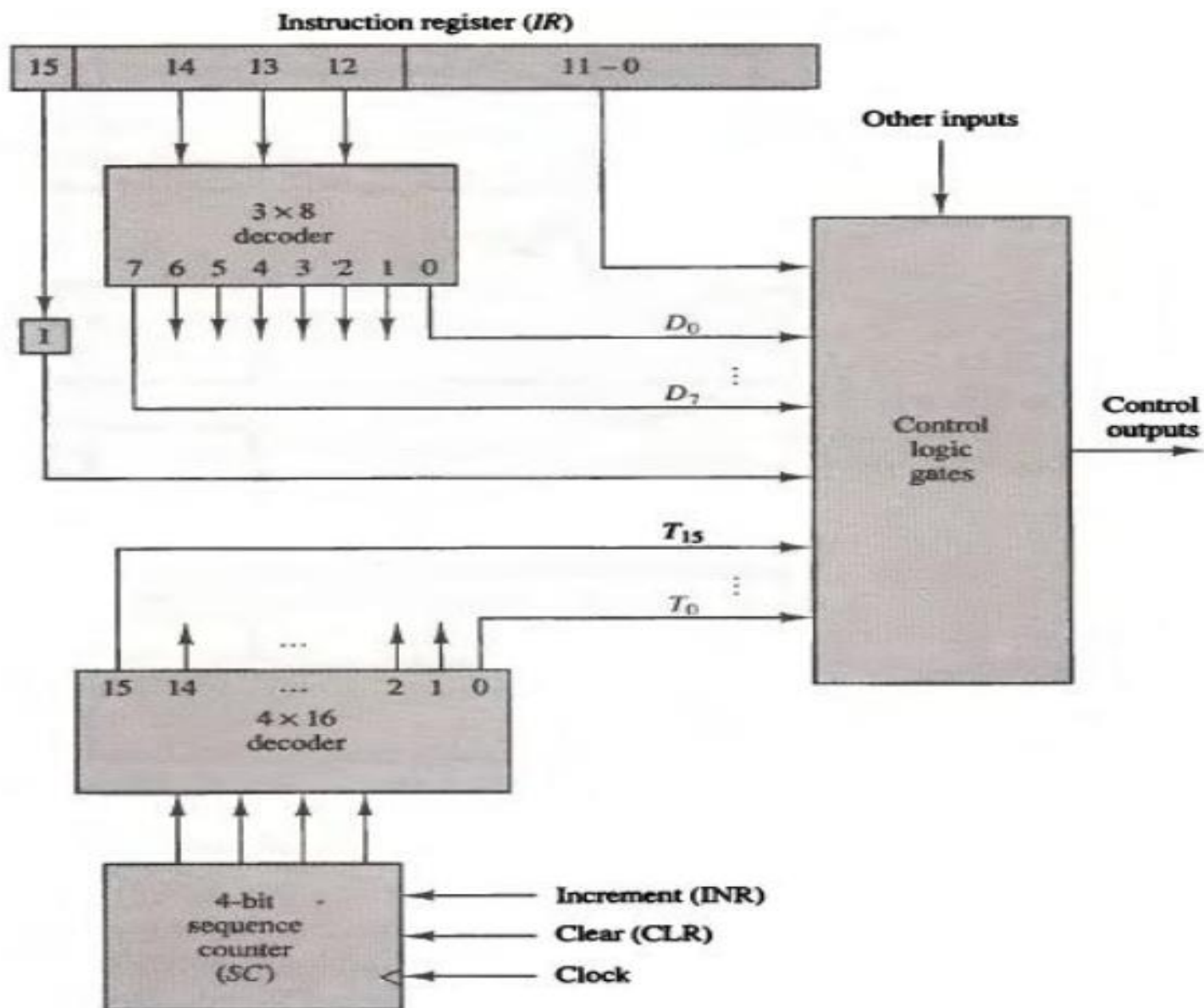
- The timing for all register of computer is controlled by a master clock generator.
- The control signal are generated in the control unit and provides control inputs for multiplexers in the common bus, control inputs in the processor registers and microoperation for accumulator.
- **There are two types of control organization:**
 - 1. Hardwired Control**
 - 2. Micro-Programmed Control**

Hardwired control:

- In this hardwired organization, the control logic is implemented with gates, flip-flops, decoders and other digital circuits.
- If the design has to be modified or changed it requires changes in the wiring among the various components.

Construction and working:

- The control unit consists of two decoders, a sequence counter and a number of control logic gates.
- A read instruction is placed in the instruction register(IR).
- In this instruction register, there are 3 parts-1 I bit, operation code, and bits 0 to 11. I bit is transferred to flip –flop ,the operation code in bits 12 through 14 are decode in D_0 - D_7 by 3:8 decoder, bits 0 to 11 is applied to control logic gates.
- Next counter decoder 4:16 provided 16 timing signals t_0 through t_{15} .
- The sequence counter(SC) can be incremented or cleared synchronously.



- **Microoperations:**

- In computer central processing units, micro-operations are detailed low-level instructions used in some designs to implement complex machine instructions (sometimes termed macro-instructions in this context).

- **Micro instruction:**

- A symbolic microprogram can be translated into its binary equivalent by means of an assembler.
- Each line of the assembly language microprogram defines a symbolic microinstruction.
- Each symbolic microinstruction is divided into five fields: label, microoperations, CD, BR, and AD.

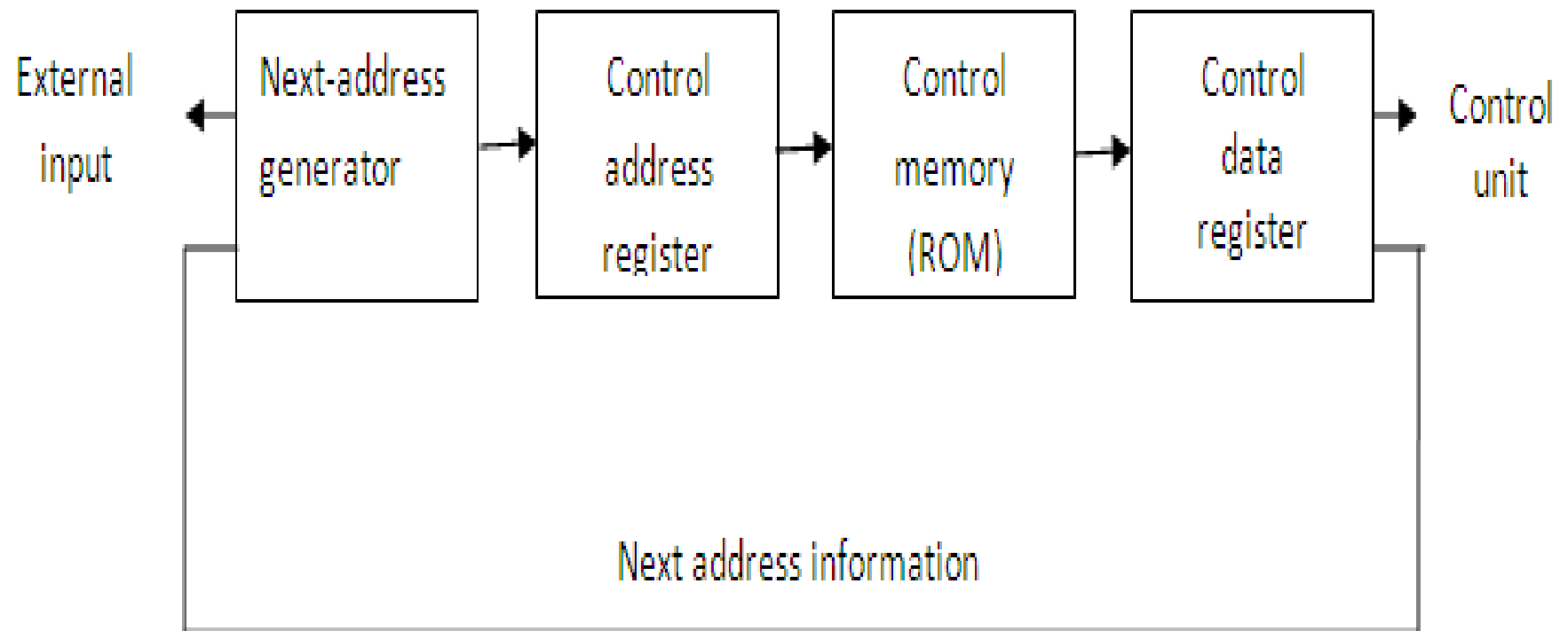
- **Micro program:**

- A sequence of microinstructions constitutes a microprogram.
- Since alterations of the microprogram are not needed once the control unit is in operation, the control memory can be a read-only memory (ROM).
- ROM words are made permanent during the hardware production of the unit.
- The use of a micro program involves placing all control variables in words of ROM for use by the control unit through successive read operations.
- The content of the word in ROM at a given address specifies a microinstruction.

Micro-programmed control:

The control memory is assumed to be a ROM, within which all control information is permanently stored

- The control memory address register specifies the address of the microinstruction, and the control data register holds the microinstruction read from memory.
- The microinstruction contains a control word that specifies one or more microoperations for the data processor. Once these operations are executed, the control must determine the next address.
- The location of the next microinstruction may be the one next in sequence, or it may be located somewhere else in the control memory.



- While the microoperations are being executed, the next address is computed in the next address generator circuit and then transferred into the control address register to read the next microinstruction.
- Thus a microinstruction contains bits for initiating microoperations in the data processor part and bits that determine the address sequence for the control memory.
- The next address generator is sometimes called a ***micro-program sequencer***, as it determines the address sequence that is read from control memory.
- Typical functions of a micro-program sequencer are incrementing the control address register by one, loading into the control address register an address from control memory, transferring an external address, or loading an initial address to start the control operations.

- The control data register holds the present microinstruction while the next address is computed and read from memory.
- The data register is sometimes called a ***pipeline register***.
- It allows the execution of the microoperations specified by the control word simultaneously with the generation of the next microinstruction.
- This configuration requires a two-phase clock, with one clock applied to the address register and the other to the data register.
- The main advantage of the micro programmed control is the fact that once the hardware configuration is established; there should be no need for further hardware or wiring changes.
- If we want to establish a different control sequence for the system, all we need to do is specify a different set of microinstructions for control memory.

Interrupt

- An interrupt is a signal to the processor emitted by hardware or software indicating an event that needs immediate attention.
- An interrupt alerts the processor to a high-priority condition requiring the interruption of the current code the processor is executing.
- The processor responds by suspending its current activities, saving its state, and executing a function called an *interrupt handler* (or an interrupt service routine, ISR) to deal with the event.

- This interruption is temporary, and, after the interrupt handler finishes, the processor resumes normal activities.
- **Interrupt are of two types**
 - (1) Hardware Interrupts**
 - (2) Software interrupt**

Hardware Interrupts

- are used by devices to communicate that they require attention from the operating system.
- hardware interrupts are implemented using electronic alerting signals that are sent to the processor from an external device,
- which is either a part of the computer itself, such as a disk controller, or an external peripheral.
- For example, pressing a key on the keyboard or moving the mouse triggers hardware interrupts that cause the processor to read the keystroke or mouse position.

External Interrupts

- External interrupts come from input-output (I/O) devices, from a timing device, from a circuit monitoring the power supply, or from any other external source.
- Examples that cause external interrupts are I/O device requesting transfer of data, I/O device finished transfer of data, elapsed time of an event, or power failure.
- External Interrupts are asynchronous.
- External Interrupts depends on external conditions that are independent of the program being executed at the time.

Two types of Hardware Interrupts

- (1) Maskable Interrupt: The hardware interrupts which can be delayed when a much high priority interrupt has occurred at the same time.
- (2) Non Maskable Interrupt: The hardware interrupts which can not be delayed and should processed by the processor immediately.

Software interrupts:

- A software interrupt is a special call instruction that behaves like an interrupt rather than a subroutine call.
- It can be used by the programmer to initiate an interrupt procedure at any desired point in the program.
- This instruction provides means for switching from a CPU user mode to the supervisor mode.
- Certain operations in the computer may be assigned to the supervisor mode only, as for example, a complex input or output transfer procedure

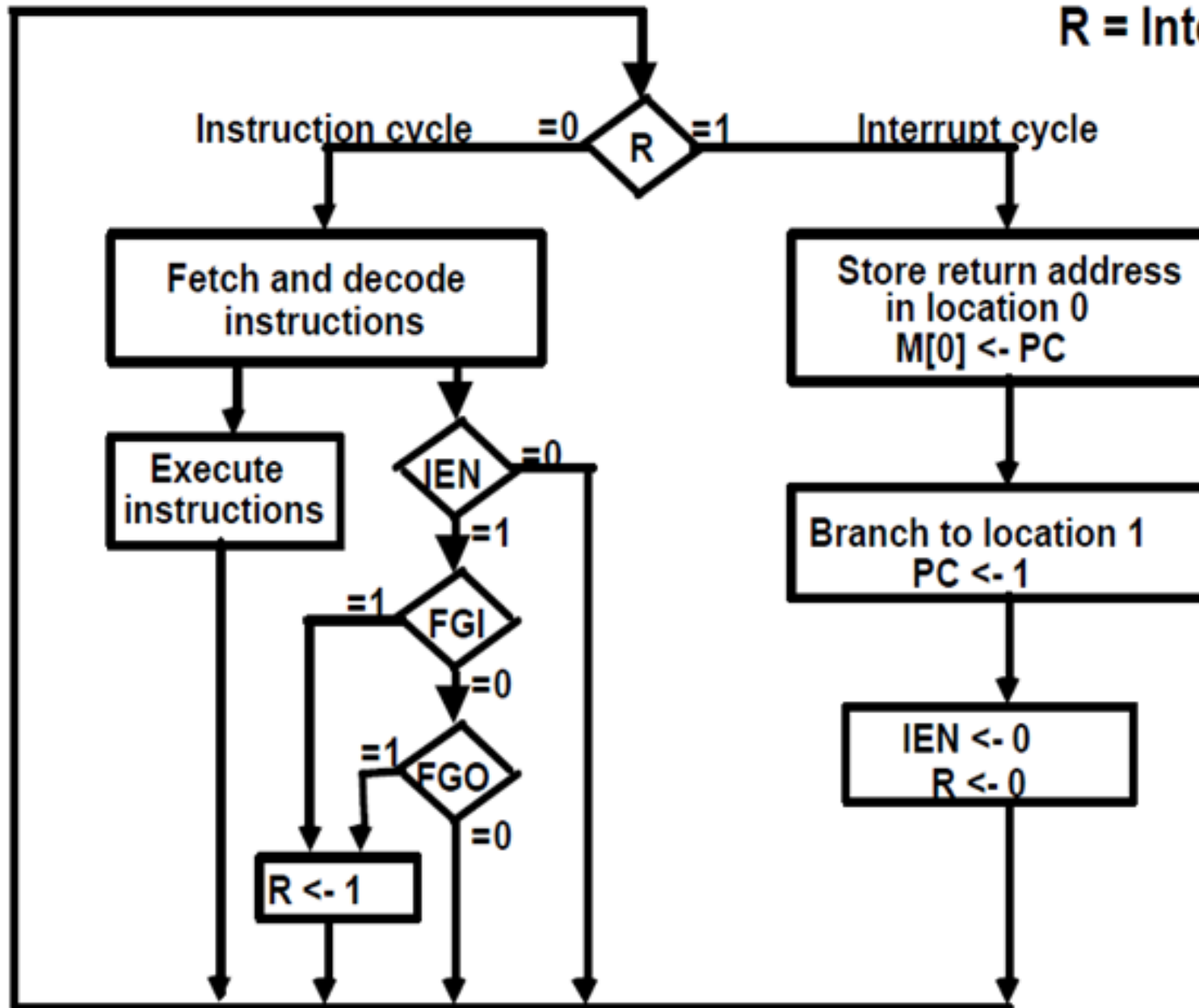
Internal Interrupts:

- Internal interrupts arise from illegal use of an instruction or data. Internal interrupts are also called traps.
- Examples of interrupts caused by internal error conditions are register overflow, attempt to divide by zero, an invalid operation code, stack overflow, and protection violation.
- These error conditions usually occur as a result of a premature termination of the instruction execution.
- Internal interrupts are synchronous with the program. . If the program is rerun, the internal interrupts will occur in the same place each time.

- **Interrupt cycle:**
- The interrupt cycle is added to the instruction cycle.
- Processor checks for interrupt indicated by an interrupt flag.
- If there is NO interrupt → Fetch next instruction

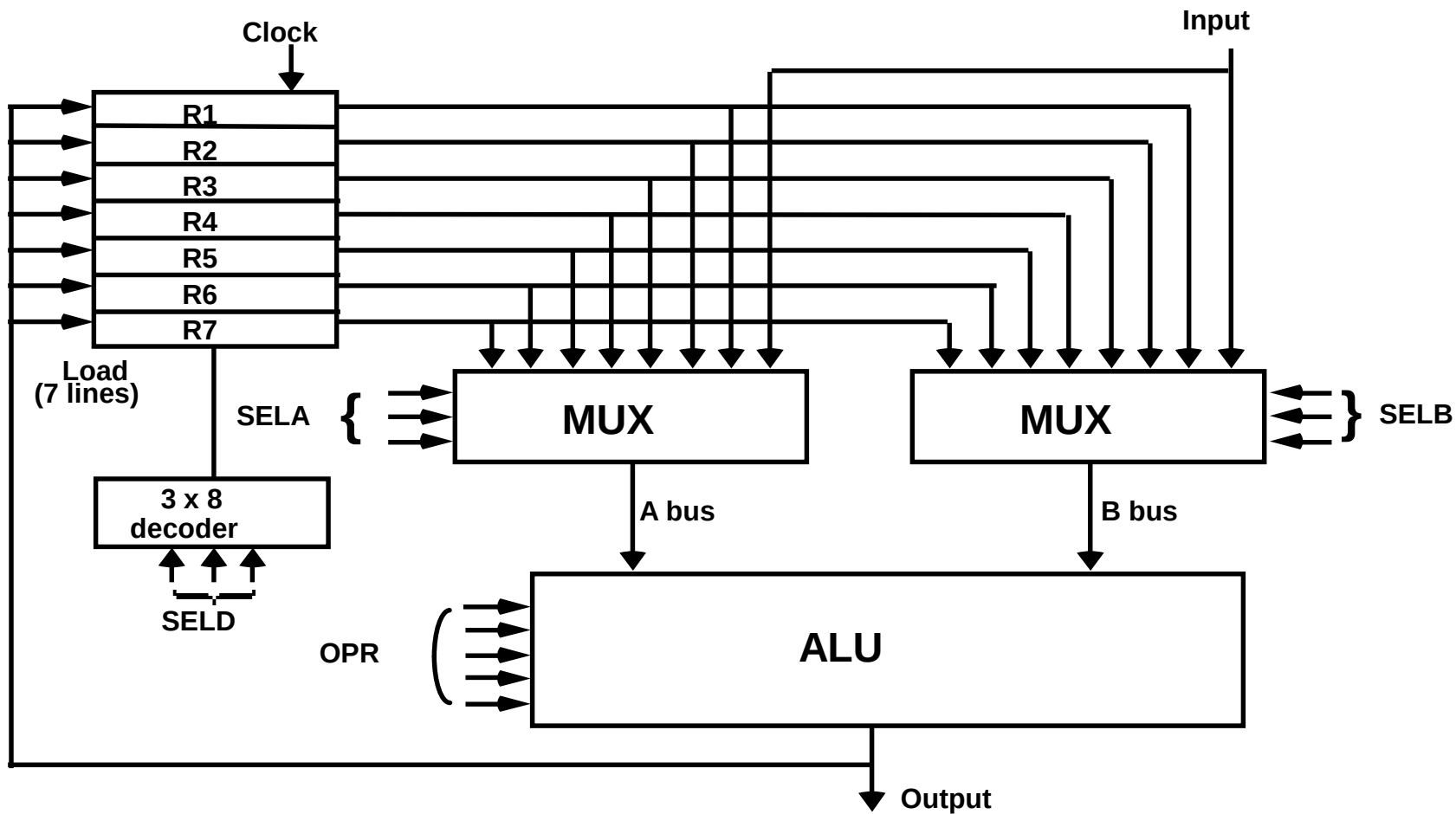
- If there is an interrupt:
 - a. Suspend operation of the program
 - b. Save its context
 - c. Set PC to start address of the interrupt handler
 - d. Process the interrupt
 - e. Restore the context of the original program and continue its execution.

R = Interrupt f/f



GENERAL REGISTER ORGANIZATION

- Generally CPU has seven general registers.
- Register organization show how registers are selected and how data flow between register and ALU.
- A decoder is used to select a particular register.
- The output of each register is connected to two multiplexers to form the two buses A and B.
- The selection lines in each multiplexer select the input data for the particular bus.



- The A and B buses form the two inputs of an ALU.
- The operation select lines decide the micro operation to be performed by ALU.
- The result of the micro operation is available at the output bus.
- The output bus connected to the inputs of all registers, thus by selecting a destination register it is possible to store the result in it.

OPERATION OF CONTROL UNIT

The control unit directs the information flow through ALU by:

- Selecting various *Components* in the system
- Selecting the *Function* of ALU

Example: $R1 \leftarrow R2 + R3$

[1] MUX A selector (SELA): $BUS\ A \leftarrow R2$

[2] MUX B selector (SELB): $BUS\ B \leftarrow R3$

[3] ALU operation selector (OPR): ALU to ADD

[4] Decoder destination selector (SELD): $R1 \leftarrow Out\ Bus$

Control Word

SELA	SELB	SELD	OPR
------	------	------	-----

Encoding of register selection fields

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

ALU CONTROL

Encoding of ALU operations

OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	ADD A + B	ADD
00101	Subtract A - B	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA

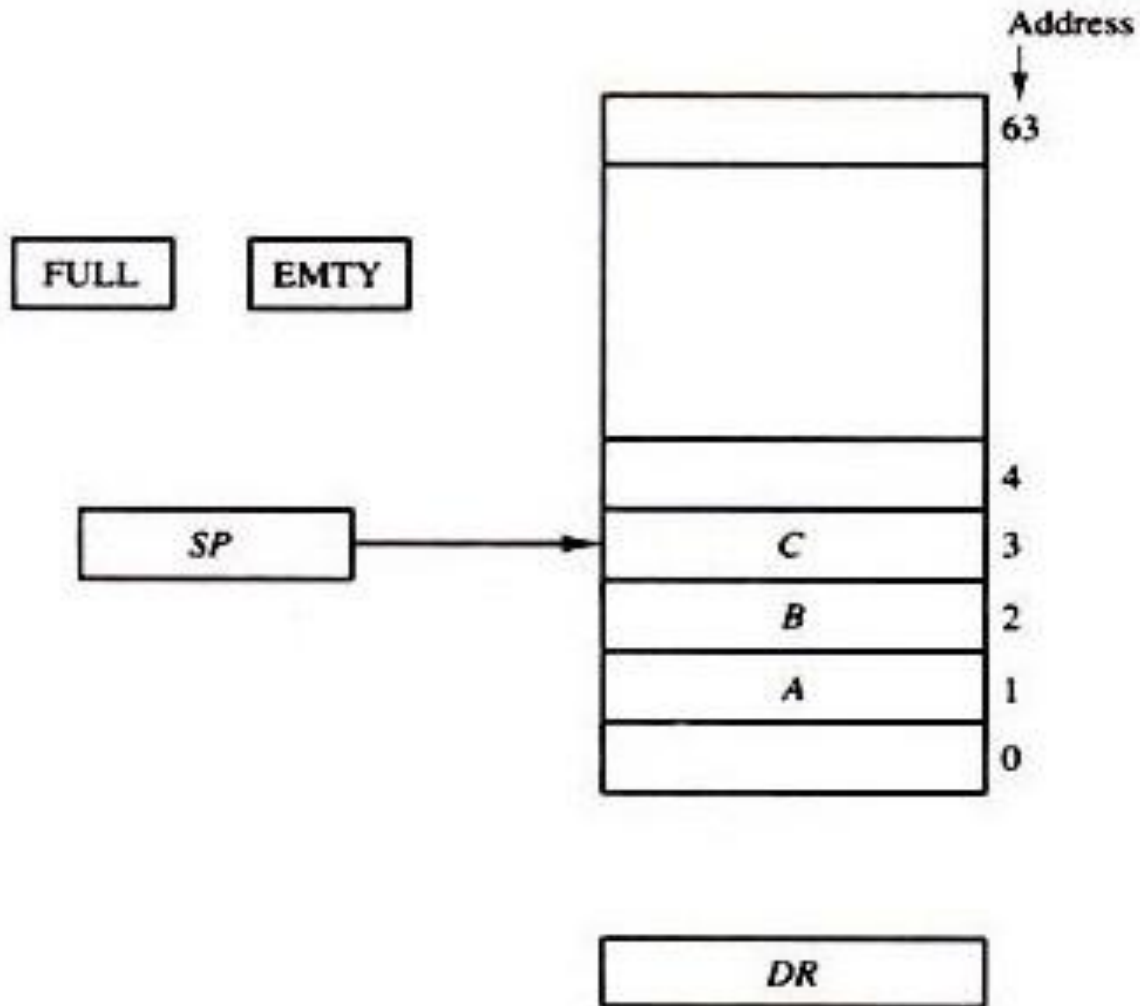
Examples of ALU Microoperations

Microoperation	Symbolic Designation				Control Word			
	SELA	SELB	SELD	OPR				
$R1 \leftarrow R2 - R3$	R2		R3	R1 SUB	010	011	001	00101
$R4 \leftarrow R4 \vee R5$	R4		R5	R4 OR	100	101	100	01010
$R6 \leftarrow R6 + 1$	R6	-	R6	INCA	110	000	110	00001
$R7 \leftarrow R1$	R1	-	R7	TSFA	001	000	111	00000
$\text{Output} \leftarrow R2$	R2		-	None TSFA	010	000	000	00000
$\text{Output} \leftarrow \text{Input}$	Input		-	None TSFA	000	000	000	00000
$R4 \leftarrow \text{shl } R4$	R4	-	R4	SHLA	100	000	100	11000
$R5 \leftarrow 0$	R5	R5	R5	XOR	101	101	101	01100

Stack Organization

- A stack is a storage device that stores information in such a manner that the item stored last is the first item retrieved(LIFO).
- The register that holds the address for the stack is called a stack pointer (SP) because its value always points at the top item in the stack.
- There are two types of stack organization
 - 1.Register Stack-Built using registers
 - 2.Memory Stack-Logical part of memory allocated as stack

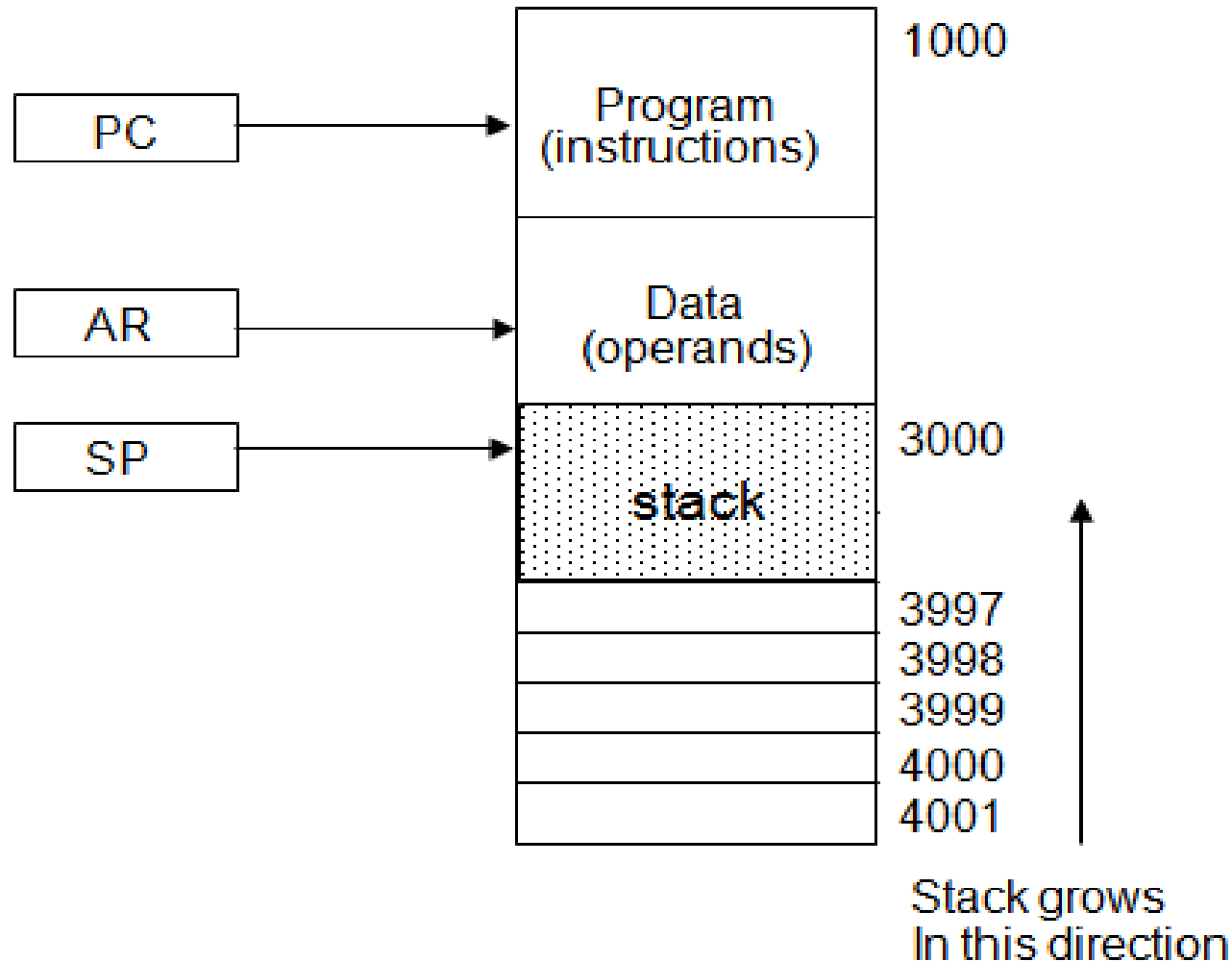
Register Stack



- ***PUSH:***
 - If the stack is not full ($FULL = 0$), a new item is inserted with a push operation. The push operation consists of the following sequences of microoperations:
- $SP \leftarrow SP + 1$ Increment stack pointer
- $M[SP] \leftarrow DR$ Write item on top of the stack
- IF ($SP = 0$) then ($FULL \leftarrow 1$) Check if stack is full
- $EMPTY \leftarrow 0$ Mark the stack not empty

- ***POP:***
 - A new item is deleted from the stack if the stack is not empty (if $EMPTY = 0$). The pop operation consists of the following sequences of microoperations:
- $DR \leftarrow M[SP]$ Read item on top of the stack
- $SP \leftarrow SP - 1$ Decrement stack pointer
- IF $(SP = 0)$ then $(EMPTY \leftarrow 1)$ Check if stack is empty
- $FULL \leftarrow 0$ Mark the stack not full

Memory Stack



- ***PUSH***
 - A new item is inserted with the push operation as follows:
- $SP \leftarrow SP - 1$
- $M[SP] \leftarrow DR$
 - The stack pointer is decremented so that it points at the address of the next word.
 - A memory write operation inserts the word from DR into the top of the stack.
-

- ***POP***
 - A new item is deleted with a pop operation as follows:
- $DR \leftarrow M[SP]$
- $SP \leftarrow SP + 1$
 - The top item is read from the stack into DR.
 - The stack pointer is then incremented to point at the next item in the stack.
 - The two microoperations needed for either the push or pop are (1) an access to memory through SP, and (2) updating SP.
 - Which of the two microoperations is done first and whether SP is updated by incrementing or decrementing depends on the organization of the stack.

Reverse Polish Notation(RPN)

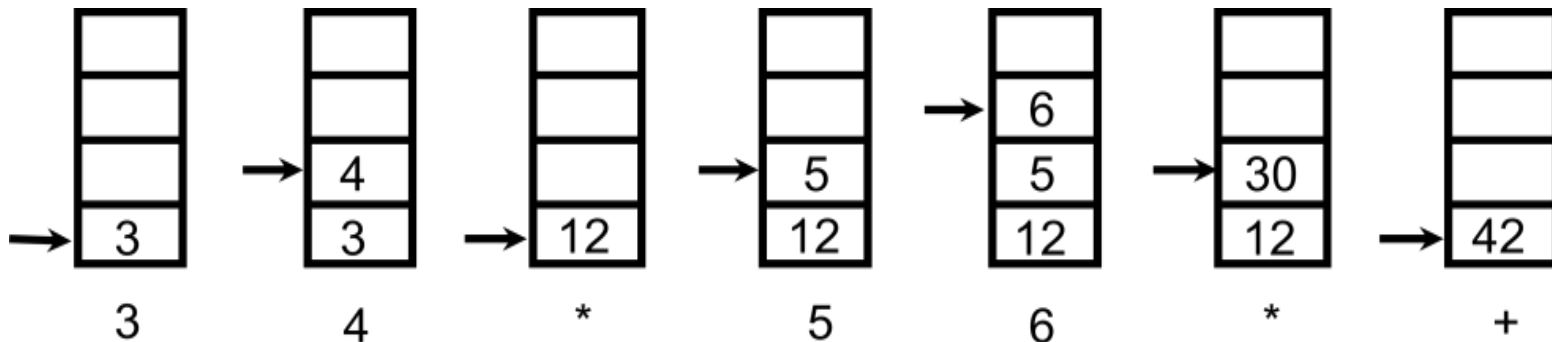
- The postfix RPN notation, referred to as Reverse Polish Notation (RPN), places the operator after the operands.
- The following examples demonstrate the three representations:
 - $A + B$ Infix notation
 - $+ A B$ Prefix or Polish notation
 - $A B +$ Postfix or reverse Polish notation
- The reverse Polish notation is in a form suitable for stack manipulation. The expression

- $A * B + C * D$ is written in reverse Polish notation as
 - $A B * C D * +$
-
- The conversion from infix notation to reverse Polish notation must take into consideration the operational hierarchy adopted for infix notation.
 - This hierarchy dictates that we first perform all arithmetic inside inner parentheses, then inside outer parentheses, and do multiplication and division operations before addition and subtraction operations.

- ***Evaluation of Arithmetic Expressions***

- Any arithmetic expression can be expressed in parenthesis-free Polish notation, including reverse Polish notation

- $(3 * 4) + (5 * 6) = 3 \ 4 \ * \ 5 \ 6 \ * \ +$



Instruction formats

- Three Address Instructions
- Two Address Instructions
- One Address Instructions
- Zero Address Instructions

- **Three Address Instructions:**

- Computers with three-address instruction formats can use each address field to specify either a processor register or a memory operand.
- The program in assembly language that evaluates $X = (A + B) * (C + D)$ is shown below.

- `ADD R1, A, B` $R1 \leftarrow M[A] + M[B]$
- `ADD R2, C, D` $R2 \leftarrow M[C] + M[D]$
- `MUL X, R1, R2` $M[X] \leftarrow R1 * R2$

- **Two Address Instructions:**

- Two address instructions are the most common in commercial computers. Here again each address field can specify either a processor register or a memory word.
- The program to evaluate $X = (A + B) * (C + D)$ is as follows:

- | | |
|--------------|---------------------------|
| • MOV R1, A | $R1 \leftarrow M[A]$ |
| • ADD R1, B | $R1 \leftarrow R1 + M[B]$ |
| • MOV R2, C | $R2 \leftarrow M[C]$ |
| • ADD R2, D | $R2 \leftarrow R2 + M[D]$ |
| • MUL R1, R2 | $R1 \leftarrow R1 * R2$ |
| • MOV X, R1 | $M[X] \leftarrow R1$ |

One Address Instructions:

- One address instructions use an implied accumulator (AC) register for all data manipulation. For multiplication and division there is a need for a second register.
- However, here we will neglect the second register and assume that the AC contains the result of all operations. The program to evaluate $X = (A + B) * (C + D)$ is

•	LOAD	A	AC <- M [A]
•	ADD	B	AC <- AC + M [B]
•	STORE	T	M [T] <- AC
•	LOAD	C	AC <- M [C]
•	ADD	D	AC <- AC + M [D]
•	MUL	T	AC <- AC * M [T]
•	STORE	X	M [X] <- AC

Zero Address Instructions:

- A stack-organized computer does not use an address field for the instructions ADD and MUL.
- The PUSH and POP instructions, however, need an address field to specify the operand that communicates with the stack.
- The program to evaluate $X = (A + B) * (C + D)$ will be written for a stack-organized computer.
- To evaluate arithmetic expressions in a stack computer, it is necessary to convert the expression into reverse polish notation.

▪ PUSH	A	TOS <- A
▪ PUSH	B	TOS <- B
▪ ADD		TOS <- (A + B)
▪ PUSH	C	TOS <- B
▪ PUSH	D	TOS <- D
▪ ADD		TOS <- (C + D)
▪ MUL		TOS <- (C + D) * (A + B)
▪ POP	X	M [X] <- TOS

Addressing Modes

- The techniques for specifying the address of the operands are known as addressing modes.
- The address of an operand is known as effective address.
- Types of addressing modes

- **Implied mode:**

- In this mode the operands are specified implicitly in the definition of the instruction.
- For example, the instruction “complement accumulator” is an implied-mode because the operand in the accumulator register is implied in the definition of the instruction.
- Example:-CMA:Take complement of the content of AC

- **Immediate Mode:**

- In this mode the operand is specified in the instruction itself. In other words, an immediate-mode instruction has an operand field rather than an address field.
- The operand field contains the actual operand to be used in conjunction with the operation specified in the instruction.
- Immediate mode of instructions is useful for initializing register to constant value.
- Example: ADD 05

- **Register Mode:**

- In this mode the operands are in registers that within the CPU. The particular register is selected from a register field in the instruction.
- Example: MOV R1,R2

-

- **Register Indirect Mode:**

- In this mode the instruction specifies a register in the CPU whose contents give the address of the operand in memory.
- The advantage of this mode is that address field of the instruction uses fewer bits to select a register than would have been required to specify a memory address directly.
- Example: ADD R1, (R2)

- **Direct Address Mode:**

- In this mode the effective address is equal to the address part of the instruction.
- The operand resides in memory and its address is given directly by the address field of the instruction.
- Example: LD ADR AC <- M[ADR]

- **Indirect Address Mode:**

- In this mode the address field of the instruction gives the address where the effective address is stored in memory.
- Control fetches the instruction from memory

- and uses its address part to access memory again to read the effective address.
- The effective address in this mode is obtained from the following computational:
- Effective address = address part of instruction + content of CPU register
- Example: $LD@ADR \ AC \leftarrow M[M[ADR]]$

- **Relative Address Mode:**

- In this mode the content of the program counter is added to the address of the instruction in order to obtain the effective address.
- When this number is added to the content of the program counter, the result produces an effective address whose position in memory is relative to the address of the next instruction.

Example: LD\$ ADR AC <- M[PC+ADR]

- **Indexed Addressing Mode:**

- In this mode the content of an index register is added to the address part of the instruction to obtain the effective address.
- The indexed register is a special CPU register that contains an index value.

– Example:-LD ADR (X) AC <- M[ADR +XR]

- **Autoincrement or Autodecrement Mode:**

- This is similar to the register indirect mode expect that the register is incremented or decremented after (or before) its value is used to access memory.
- When the address stored in the register refers to a table of data in memory, it is necessary to increment or decrement the register after every access to the table. This can be achieved by using the increment or decrement instruction.
- Example of Auto increment : Add R1, (R2)+
- Example of Auto decrement: Add R1, -(R2)

RISC

- It stands for Reduced Instruction Set Computer.
- It is a type of microprocessor architecture that uses a small set of instructions of uniform length.
- These are simple instructions which are generally executed in one clock cycle.
- RISC chips are relatively simple to design and inexpensive.
- The setback of this design is that the computer has to repeatedly perform simple operations to execute a larger program having a large number of processing operations.

-

Examples: SPARC, POWER PC etc.

CISC

- It stands for Complex Instruction Set Computer.
- These processors offer the users, hundreds of instructions of variable sizes.
- CISC architecture includes a complete set of special purpose circuits that carry out these instructions at a very high speed.
- These instructions interact with memory by using complex addressing modes.
- CISC processors reduce the program size and hence lesser number of memory cycles are required to execute the programs. This increases the overall speed of execution.
- **Examples:** Intel architecture, AMD

RISC vs. CISC

CISC	RISC
Emphasis on hardware	Emphasis on software
Multiple instruction sizes and formats	Instructions of same set with few formats
Less registers	Uses more registers
More addressing modes	Fewer addressing modes
Extensive use of microprogramming	Complexity in compiler
Instructions take a varying amount of cycle time	Instructions take one cycle time
Pipelining is difficult	Pipelining is easy

Characteristics of memory systems

- The memory system can be characterised with their Location, Capacity, Unit of transfer,
- Access method, Performance, Physical type, Physical characteristics, Organisation.

- **Location**

- • Processor memory: The memory like registers is included within the processor and
 - termed as processor memory.
- • Internal memory: It is often termed as main memory and resides within the CPU.
- • External memory: It consists of peripheral storage devices such as disk and magnetic
 - tape that are accessible to processor via i/o controllers.

- **Capacity**

- • Word size: Capacity is expressed in terms of words or bytes.
- — The natural unit of organisation
- • Number of words: Common word lengths are 8, 16, 32 bits etc.
- — or Bytes

- **Unit of Transfer**

- • Internal: For internal memory, the unit of transfer is equal to the number of data lines into and out of the memory module.
- • External: For external memory, they are transferred in block which is larger than a word.

- **Access Method**

- • Sequential access— e.g. tape
- • Direct Access:— e.g. disk
- • Random access:— e.g. RAM

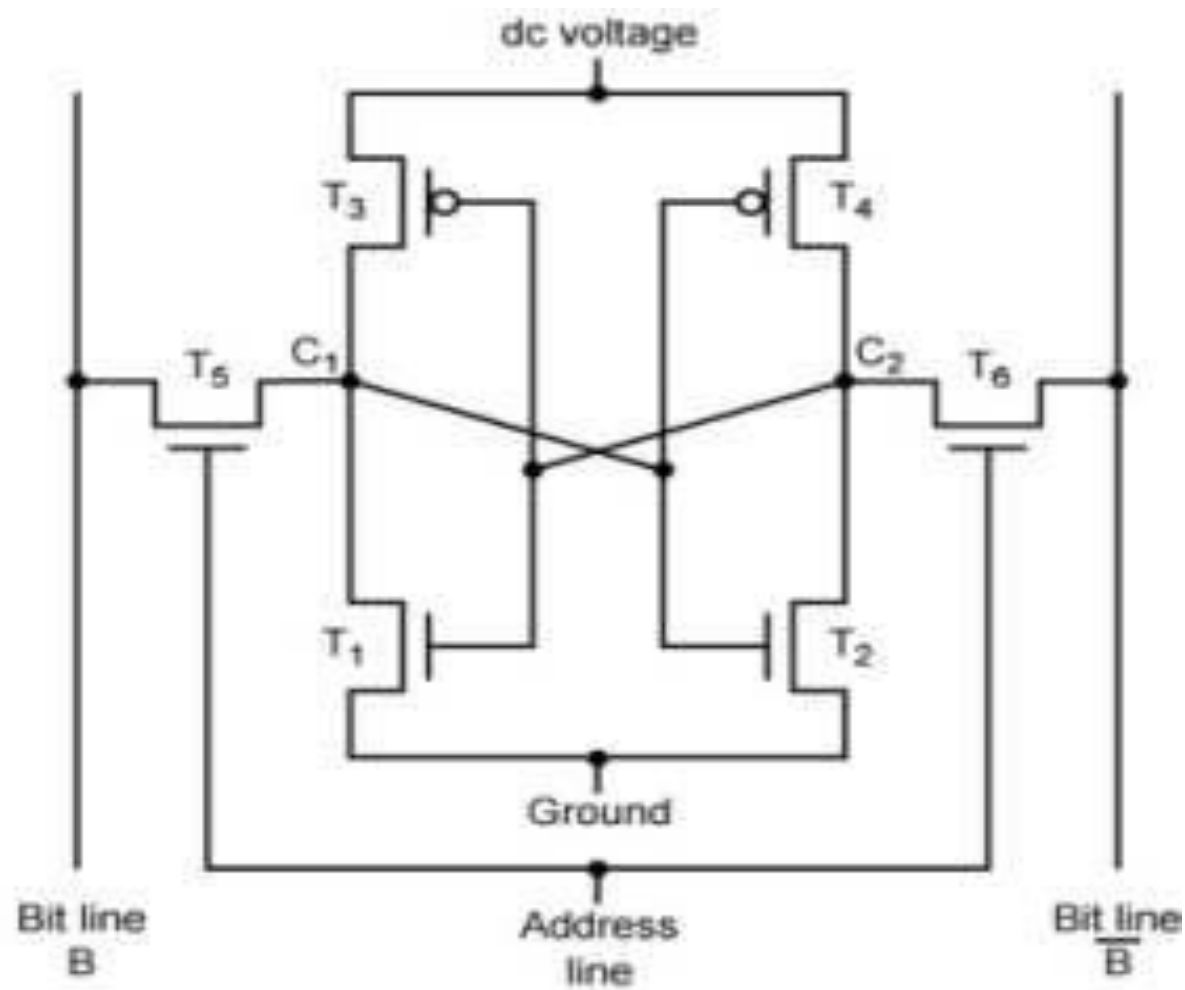
- **Performance**
- • Access time:
- • Memory Cycle time
- Memory cycle time = access time plus transient time (any additional time required)
- • Transfer Rate:

RAM– Random Access memory

- Memory cells can be accessed for information transfer from any desired random location.
- The process of locating a word in memory is the same and requires of locating
- A word in memory is the same and requires an equal amount of time no matter where the cells are located physically in memory thus named 'Random.
- RAM are available in two possible operating modes, *Static and Dynamic*.

Static RAM (SRAM)

- The static RAM consists of flip flop that stores binary information and this stored information remains valid as long as power is applied to the unit.

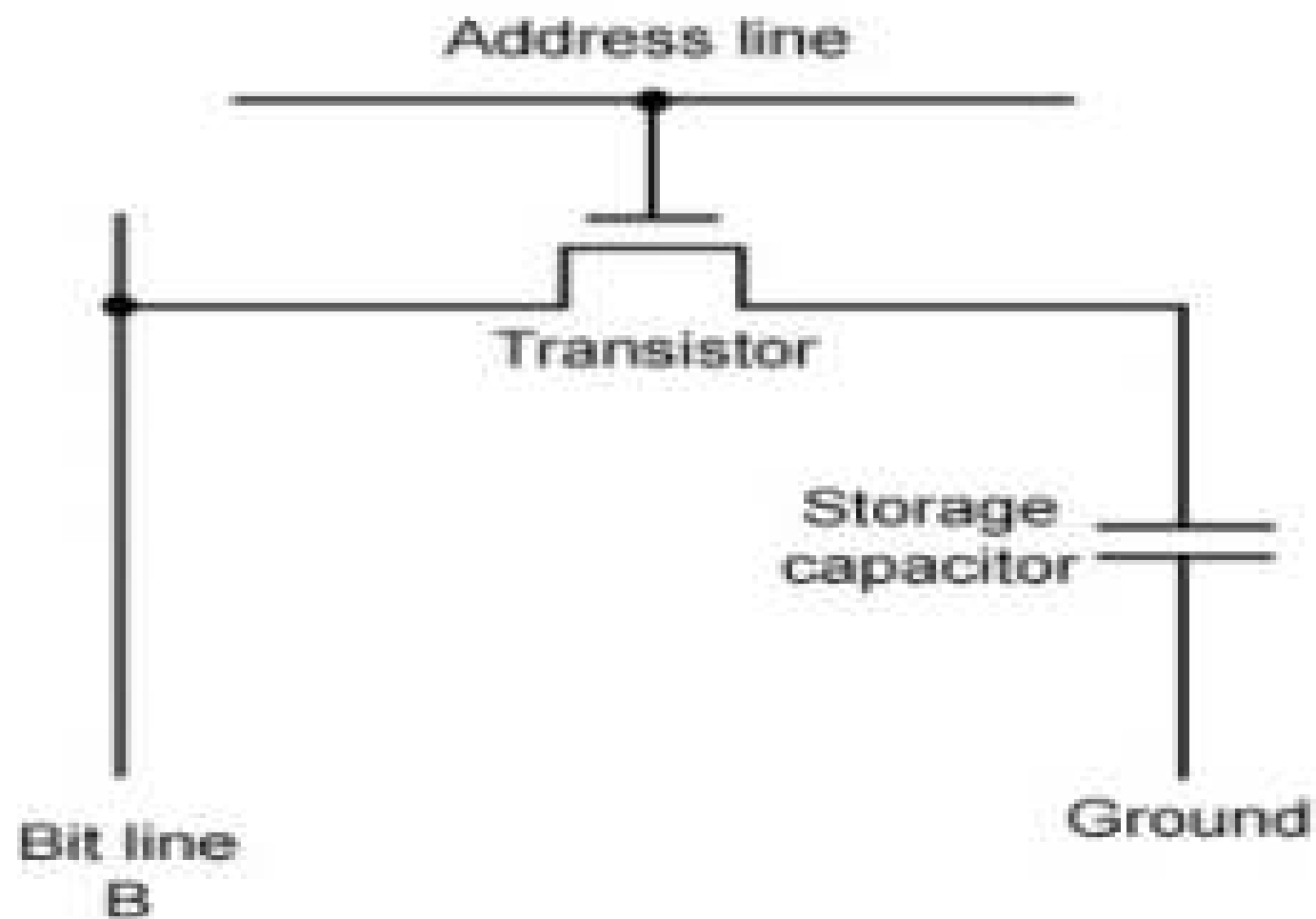


- Four transistors T1, T2, T3 and t4 are cross connected in an arrangement that produces a stable logical state.
- In logic state 1, point C1 is high and point C2 is low. In this state, T1 & T4 are off and T2 & T3 are on.
- In logic state 0, point C1 is low and C2 is high. In this state, T1 & T4 are on and T2 & T3 are off.

- The address line controls the two transistors T5 & T6. When a signal is applied to this line, the two transistors are switched on allowing for read and write operation.
- For a write operation, the desired bit value is applied to line B while its complement is applied to line B complement. This forces the four transistors T1, T2, T3 & T4 into a proper state.
- For the read operation, the bit value is read from line B.

Dynamic RAM (DRAM)

- The dynamic RAM stores the binary information in the form of electrical charges and capacitor is used for this purpose.
- Since charge stored in capacitor discharges with time, capacitor must be periodically recharged and which is also called refreshing memory.



- The address line is activated when the bit value from this cell is to be read or written.
- The transistor acts as switch that is closed i.e. allowed current to flow, if voltage is applied to the address line; and opened i.e. no current to flow, if no voltage is present in the address line.

SRAM versus DRAM

- Static RAM
- Uses flip flop to store information
- Needs more space
- Faster, digital device
- Expensive, big in size
- Don't require refreshing circuit
- Used in cache memory

Dynamic RAM

- Uses capacitor to store information
- More dense i.e. more cells can be accommodated per unit area
- Slower, analog device
- Less expensive, small in size
- Needs refreshing circuit
- Used in main memory, larger memory units

ROM– Read Only memory

- Read only memory (ROM) contains a permanent pattern of data that cannot be changed.
- A ROM is non-volatile that is no power source is required to maintain the bit values in memory.
- A ROM is created like any other integrated circuit chip, with the data actually wired into the chip as part of the fabrication process.

Types of ROM

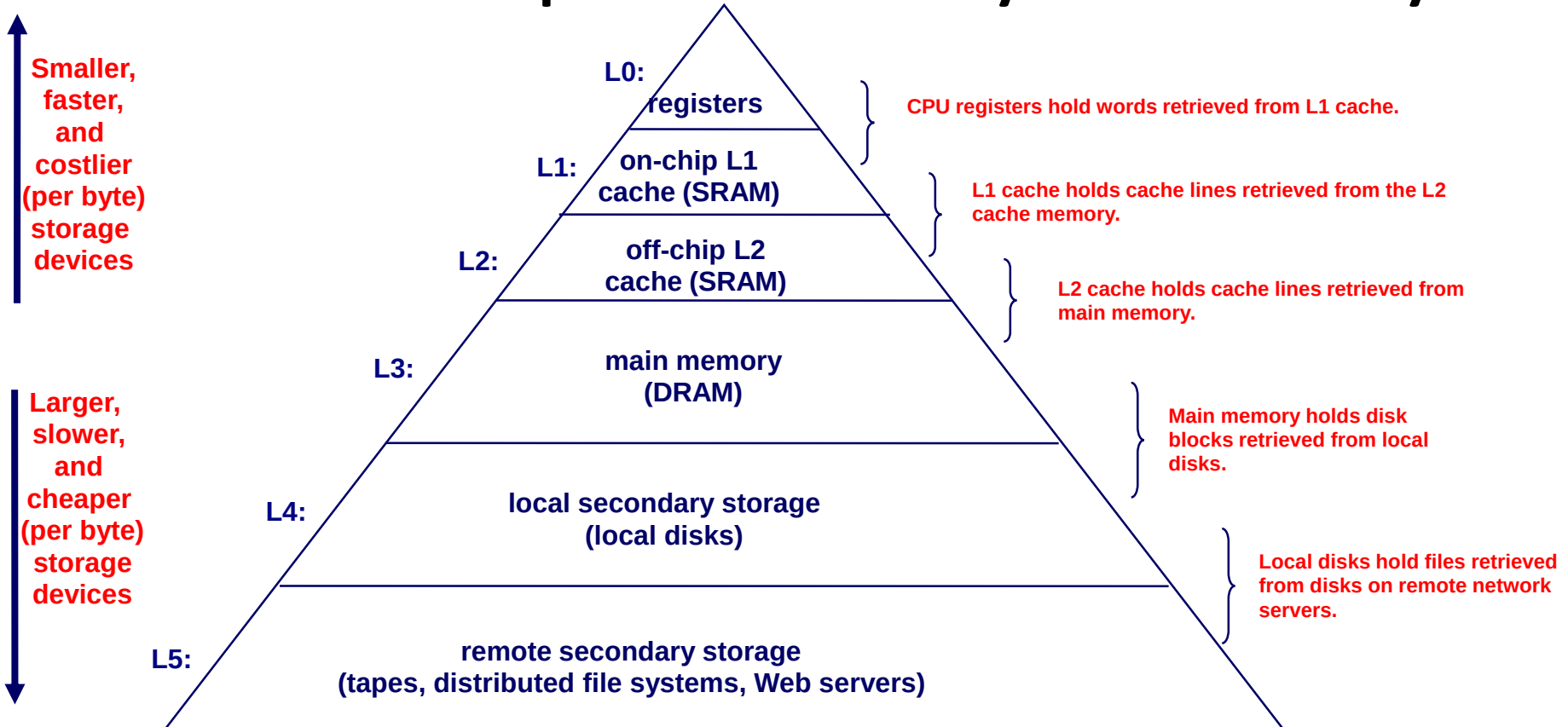
- Programmable ROM
- Erasable Programmable ROM (EPROM)
- Electrically Erasable programmable ROM (EEPROM)
- Flash Memory

Memory Hierarchy

- In computer architecture the **memory hierarchy** is a concept used to discuss performance issues in computer architectural design, algorithm predictions, and lower level programming constructs involving locality of reference.
- The memory hierarchy in computer storage separates each of its levels based on response time.
- Since response time, complexity, and capacity

- **Primary Memory** The primary memory is also known as internal memory, and this is accessible by the processor . This memory includes main, cache, as well as CPU registers.
- **Secondary Memory** The secondary memory is also known as external memory, and this is accessible by the processor through an input/output module. This memory includes an optical disk, magnetic disk, and magnetic tape.

An Example Memory Hierarchy



From lecture-9.ppt

Characteristics of Memory Hierarchy

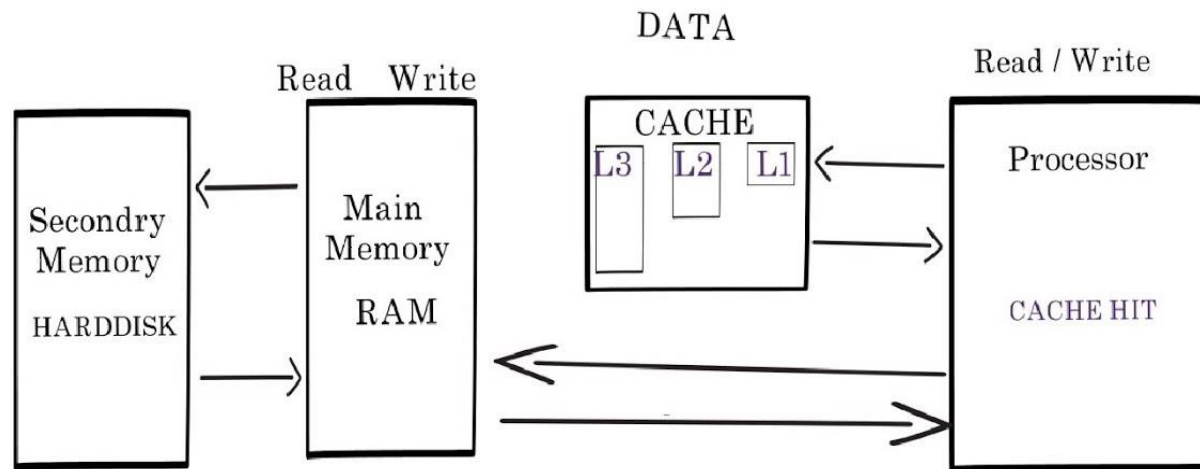
- The memory hierarchy characteristics mainly include the following.
- **Performance**
- The speed gap among the main memory as well as the CPU registers enhances because of the huge disparity in access time, which will cause the lower performance of the system.
- So, the enhancement was mandatory. The enhancement of this was designed in the memory hierarchy model due to the system's performance increase.
- **Ability**
- The ability of the memory hierarchy is the total amount of data the memory can store. Because whenever we shift from top to bottom inside the memory hierarchy, then the capacity will increase.

- **Access Time**
- The access time in the memory hierarchy is the interval of the time among the data availability as well as request to read or write. Because whenever we shift from top to bottom inside the memory hierarchy, then the access time will increase
- **Cost per bit**
- When we shift from bottom to top inside the memory hierarchy, then the cost for each bit will increase which means an internal Memory is expensive compared with external memory

Cache Memory

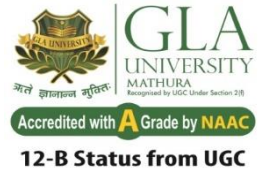
- The data or contents of the main memory that are used frequently by CPU are stored in the cache memory so that the processor can easily access that data in a shorter time.
- Whenever the CPU needs to access memory, it first checks the cache memory. If the data is not found in cache memory, then the CPU moves into the main memory.
- Cache memory is placed between the CPU and the main memory. The block diagram for a cache memory can be represented as:

CACHE MEMORY BLOCK DIAGRAM



$$\text{HIT RATIO} = \frac{\text{Nos of Hits}}{\text{Total Access}} = \frac{\text{Nos of Hits}}{\text{Nos of Hits} + \text{Nos of Miss}}$$

The basic operation of a cache memory is as follows:



- When the CPU needs to access memory, the cache is examined. If the word is found in the cache, it is read from the fast memory.
- If the word addressed by the CPU is not found in the cache, the main memory is accessed to read the word.
- A block of words one just accessed is then transferred from main memory to cache memory. The block size may vary from one word (the one just accessed) to about 16 words adjacent to the one just accessed.

- The performance of the cache memory is frequently measured in terms of a quantity called **hit ratio**.
- When the CPU refers to memory and finds the word in cache, it is said to produce a **hit**.
- If the word is not found in the cache, it is in main memory and it counts as a **miss**.
- The ratio of the number of hits divided by the total CPU references to memory (hits plus misses) is the hit ratio.

Cache mapping

- refers to the method used to store data from main memory into the cache. It determines how data from memory is mapped to specific locations in the cache.
- There are three different types of mapping used for the purpose of cache memory which is as follows:
 - Direct Mapping
 - Fully Associative Mapping
 - Set-Associative Mapping

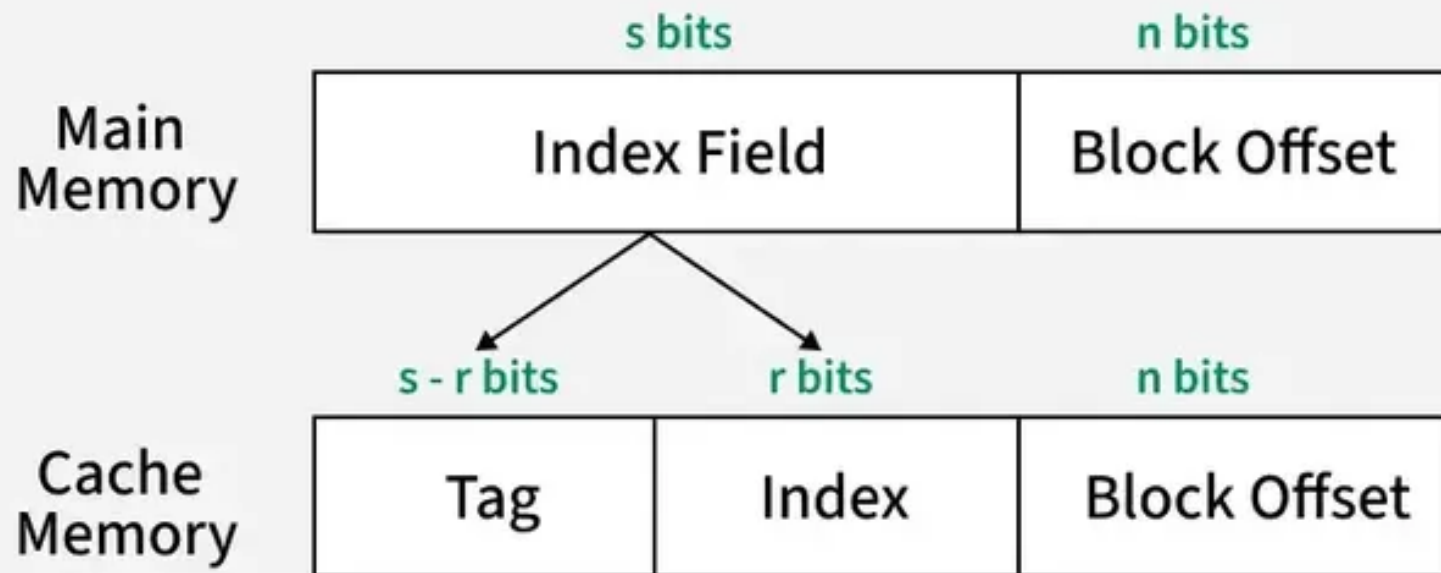
Direct Mapping

- Direct mapping is a simple and commonly used cache mapping technique where each block of main memory is mapped to exactly one location in the cache called cache line.
- If two memory blocks map to the same cache line, one will overwrite the other, leading to potential cache misses.

- Memory block is assigned to cache line using the formula below:
- $i = j \text{ modulo } m = j \% m$ where,
- i = cache line number
- j = main memory block number
- m = number of lines in the cache

- The Main Memory consists of memory blocks and these blocks are made up of fixed number of words.
- A address in main memory is split into two parts:
- Index Field: It represent the block number. Index Field bits tells us the location of block where a word can be.
- Block Offset: It represent words in a memory block. These bits determines the location of word in a memory block.

- The Cache Memory consists of cache lines. These cache lines has same size as memory blocks. The address in cache memory consists of:
- Block Offset: This is the same block offset we use in Main Memory.
- Index: It represent cache line number. This part of the memory address determines which cache line (or slot) the data will be placed in.
- Tag: The Tag is the remaining part of the address that uniquely identifies which block is currently occupying the cache line.



- The index field in main memory maps directly to the index in cache memory, which determines the cache line where the block will be stored.
- The block offset in both main memory and cache memory indicates the exact word within the block.
- In the cache, the tag identifies which memory block is currently stored in the cache line.

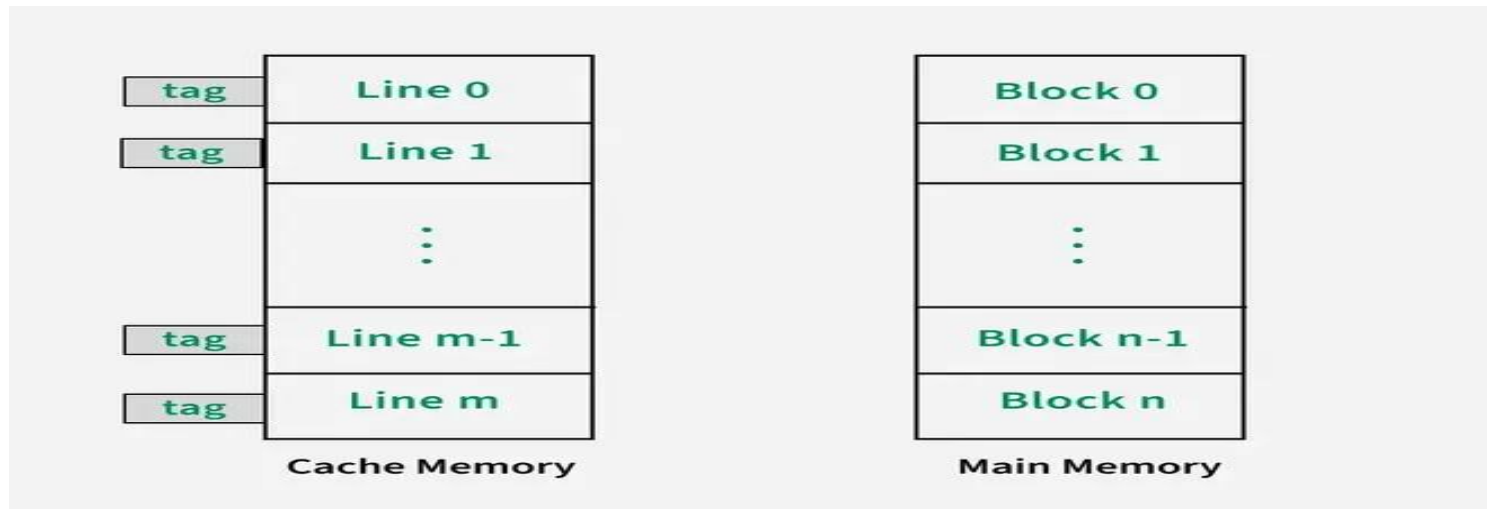
Fully Associative Mapping

- Any block of main memory can be stored in any cache line. Unlike direct-mapped cache, where each memory block is restricted to a specific cache line based on its index,
- flexibility to place a memory block in any available cache line.
- This improves the hit ratio but requires a more complex system for searching and managing cache lines.



- The address structure of Cache Memory is different in fully associative mapping from direct mapping.
- In fully associative mapping, the cache does not have an index field.
- It only have a tag which is same as Index Field in memory address.
- Any block of memory can be placed in any cache line. This flexibility means that there's no fixed position for memory blocks in the cache.

- To determine whether a block is present in the cache, the tag is compared with the tags stored in all cache lines.
- If a match is found, it is a cache hit, and the data is retrieved from that cache line.
- If no match is found, it's a cache miss, and the required data is fetched from main memory.



Set-Associative Mapping

- Set-associative mapping is a compromise between direct-mapped and fully-associative mapping in cache systems.
- It combines the flexibility of fully associative mapping with the efficiency of direct mapping.
- In this scheme, multiple cache lines (typically 2, 4, or more) are grouped into sets.

Cache
Memory

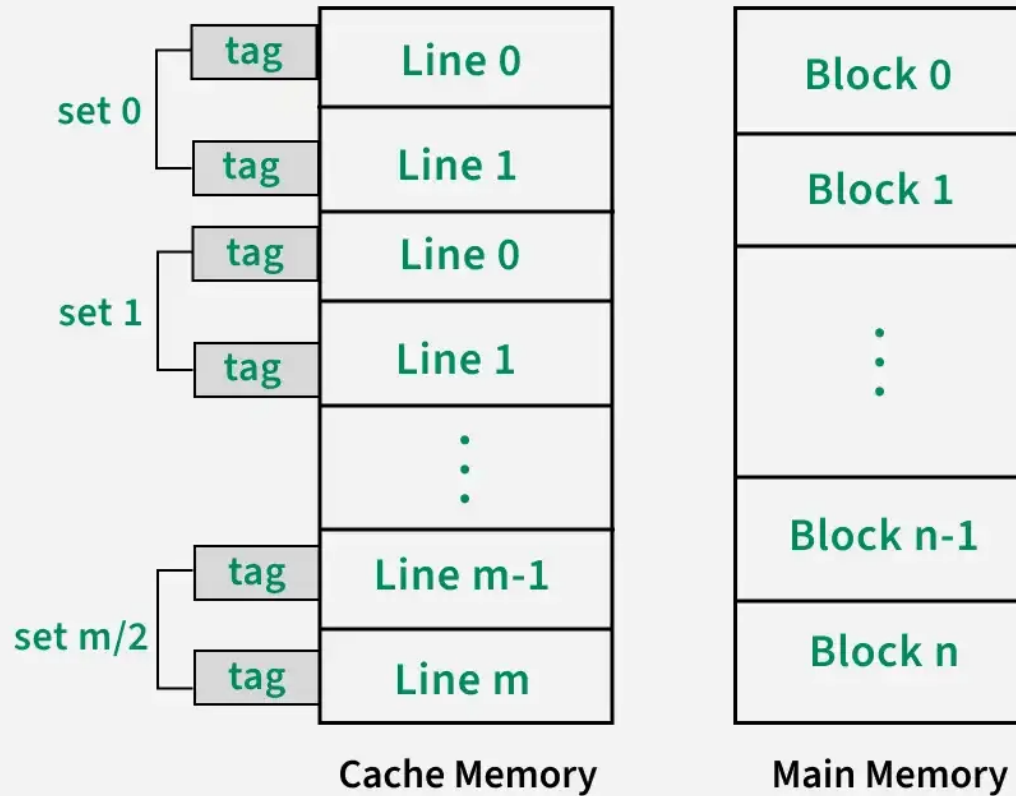
Tag

Set Number

Block Offset

- For example, consider a 2-way set associative cache, which means 2 cache lines make a set in this cache structure.
- Using direct mapping strategy first, block 0 will be in set 0, block 1 in set 1, block 2 in set 0 and so on.
- Then, the tag is used to search through all cache lines in that set to find the correct block (Associative Mapping).

Two Way Set Associative Mapping



Advantages and Disadvantages

- Cache Memory is faster in comparison to main memory and secondary memory.
- Programs stored by Cache Memory can be executed in less time.
- The data access time of Cache Memory is less than that of the main memory.
- Cache Memory stores data and instructions that are regularly used by the CPU, therefore it increases the performance of the CPU.
- Cache Memory is costlier than primary memory and secondary memory .
- Data is stored on a temporary basis in Cache Memory.
- Whenever the system is turned off, data and instructions stored in cache memory get destroyed.
- The high cost of cache memory increases the price of the Computer System.

- Consider a direct mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find-
- Number of bits in tag
- Tag directory size

Given-

Cache memory size = 16 KB

Block size = Frame size = Line size = 256 bytes

Main memory size = 128 KB

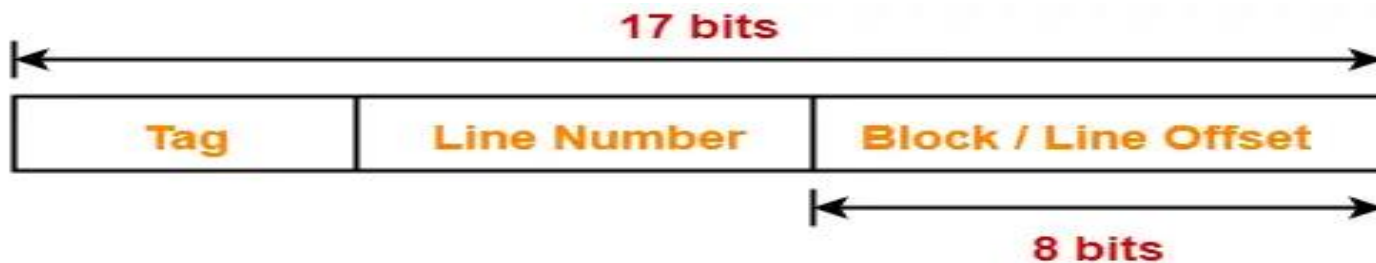
- We have,
- Size of main memory = 128 KB = 2^{17} bytes
- Thus, Number of bits in physical address = 17 bits



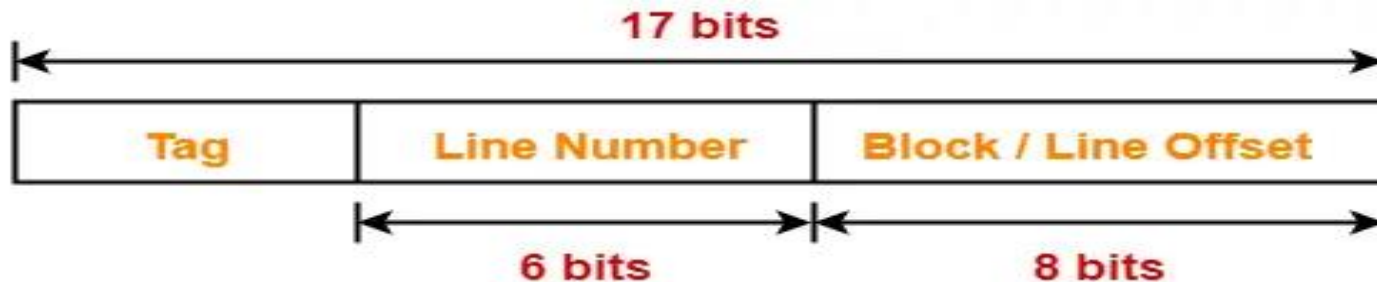
- Number of Bits in Block Offset-

We have, Block size= 256 bytes

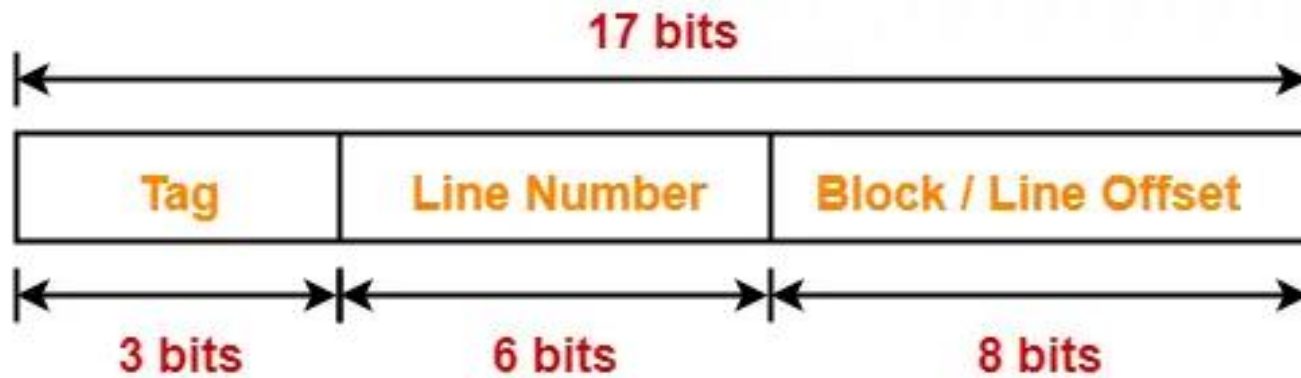
Thus, Number of bits in block offset = 8 bits



- **Number of Bits in Line Number-**
- Total number of lines in cache
- = Cache size / Line size
- = 16 KB / 256 bytes
- = 2^{14} bytes / 2^8 bytes
- = 2^6 lines
- Thus, Number of bits in line number = 6 bits



- Number of Bits in Tag-
- Number of bits in tag
- = Number of bits in physical address – (Number of bits in line number + Number of bits in block offset)
- = 17 bits – (6 bits + 8 bits)
- = 17 bits – 14 bits
- = 3 bits



- Tag Directory Size
- = Number of tags x Tag size
- = Number of lines in cache x Number of bits in tag
- = $2^6 \times 3 \text{ bits} = 192 \text{ bits} = 24 \text{ bytes}$

- A direct-mapped cache memory consists of 512 blocks. The main memory has 65,536 blocks, and each block contains 128 sixteen-bit words.
- How many bits are required to address the main memory?
- How many bits are needed to represent the TAG, INDEX, and WORD fields?

- Given Data:
- Cache Blocks = 512
- Main Memory Blocks = 65,536
- Block Size = 128 words
- Word Size = 16 bits

- Total words=Number of blocks $\times$ Words per block
- $=65,536 \times 128 = 8,388,608$ words
- Since each word in memory must have a unique address, the number of address bits required is:
- $\log_2(8,388,608) = 23$ bits
- Thus, 23 bits are required for addressing the main memory.

- Address= TAG bits + INDEX bits + WORD bits
Word Field $2^7=128$
- 7 bits are required for Word field.
- the cache has **512 blocks**, each block is uniquely identified by an index.
- Index field= $2^9=512$
- 9 bits are required for INDEX field.
- TAG bits= $23-(7+9)=7$ bits

- A block-set associative cache memory with 128 blocks are divided into four block sets. The main memory consists of 16,384 blocks where each block contains 256 eight bit words. How many bits are required for addressing the main memory?

- Total number of blocks given = **(16,384)**
- by numerical analysis = **($2^{14} = 16,384$)**
- According to the question, there are 2^{14} blocks
- and each block size is 2^8 bytes.
- thus, size of the main - memory = $2^{14} * 2^8 = 2^{22} = 4\text{MB}$
- The TAG field requires = **$22 - (5 + 8) = 9$ bits**
- thus, 9 bits for TAG, 5 bits for set, 8 bits
- as there are **4 blocks in 1 set**, **32 sets** will be needed for **128 Blocks**.
- Thus, SET field requires 5 bits.

- A block-set associative cache memory consists of 256 blocks divided into eight block sets. The main memory consists of 32,768 blocks, and each block contains 512 sixteen-bit words. How many bits are required for addressing the main memory? How many bits are needed to represent the TAG, SET, and WORD fields?

Peripheral devices

- Devices that are linked either internally or externally to a computer.
- Commonly used to transfer data.
- The most common processes that are carried out in a computer are entering data and displaying processed data.
- Several devices can be used to receive data and display processed data.
- The devices used to perform these functions are called peripherals or I/O devices.

- The most common peripherals are a printer, scanner, keyboard, mouse, tape device, microphone, and external modem that are externally connected to the computer.

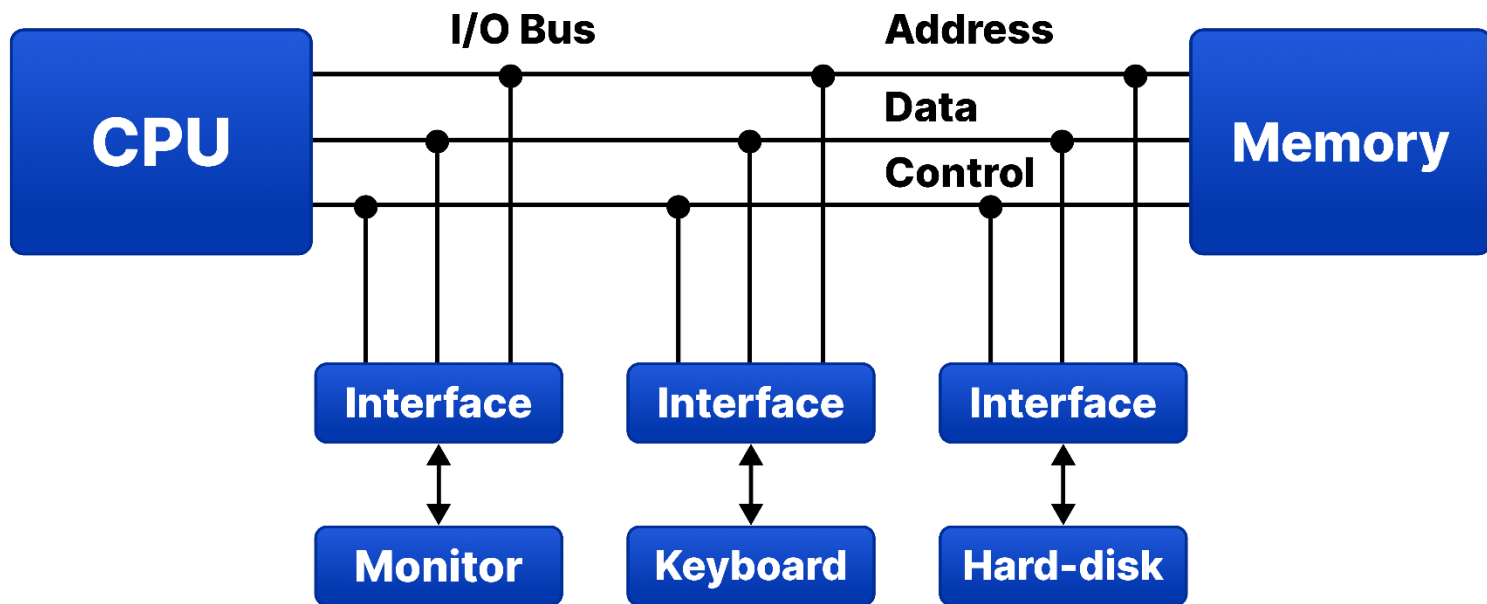
I/O interface

- Hardware component or system that manages the data transfer between the computer's internal memory or CPU and external devices.
- These devices can either be input devices (such as a keyboard or mouse) that send data to the computer, or output devices (such as a printer or monitor) that receive data from the computer.
- The I/O interface also supports serial and parallel communication, converting signals and ensuring compatibility between devices with differing data rates and formats.

Functions of Input-Output Interface

- **Speed Synchronization:** The interface ensures that the CPU's operating speed is synchronized with the input-output devices. This prevents data loss due to speed mismatches.
- **Processor Communication:** The interface accepts and decodes commands from the processor, reports the current status, and recognizes its unique address.
- **Signal Control :** It generates and manages control and timing signals needed for data transfer, ensuring smooth communication between the CPU and peripherals.

- **Data Buffering** :The interface enables buffering, which temporarily stores data as it moves between devices and the CPU, helping manage the difference in processing speeds.
- **Error Detection** :The interface can detect errors in data transmission, ensuring that errors are flagged and corrected before they affect system performance.
- **Data Conversion**: It converts serial data to parallel data and vice versa, as well as converting digital data to analog signals and vice versa, and ensures the format is compatible with the receiving device.
- **Status Reporting**: The interface reports the current status of the peripheral device to the processor.

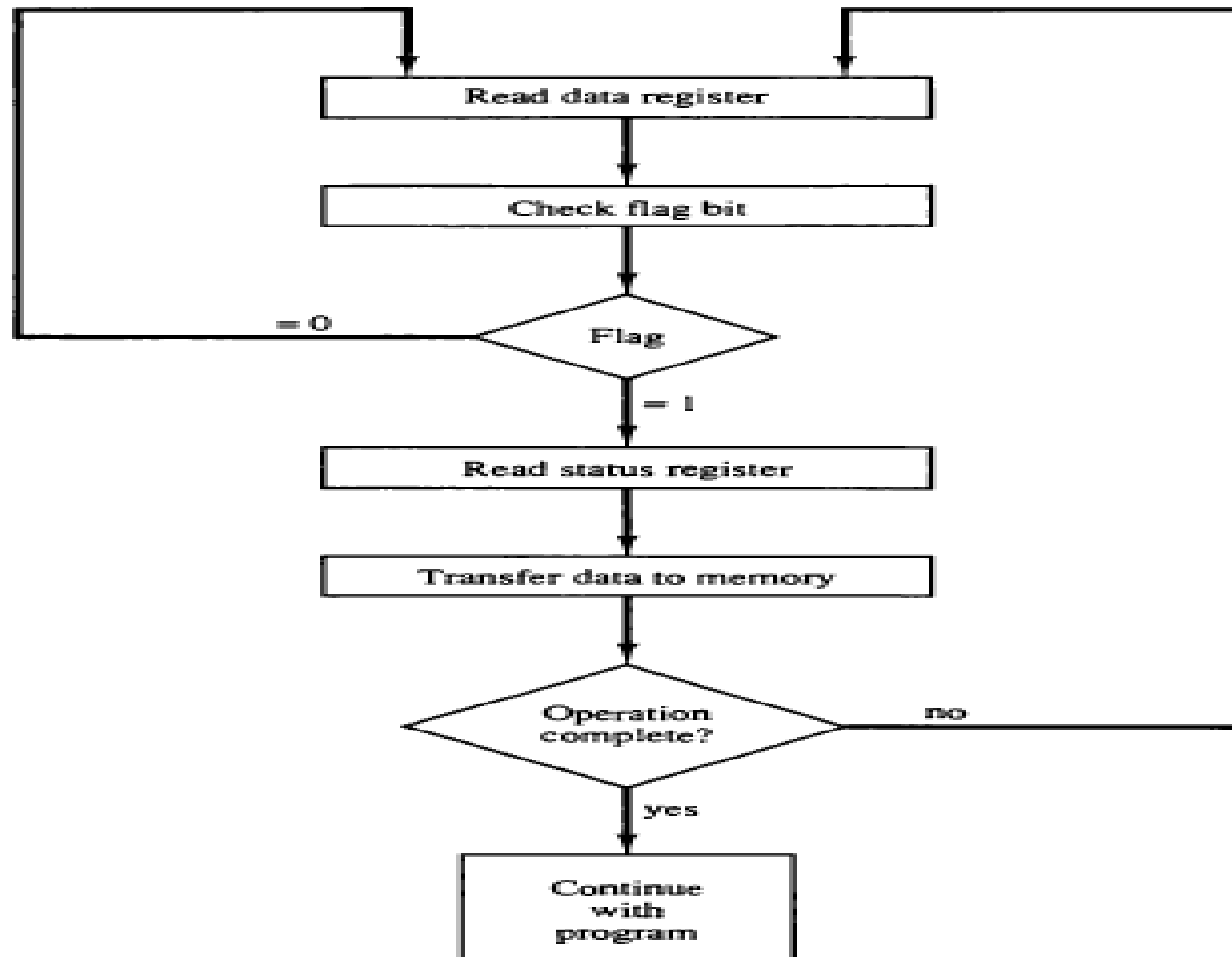


Modes of Data Transfer

- **Programmed I/O (PIO)**
- **Interrupt-Driven I/O**
- **Direct Memory Access (DMA)**

Programmed I/O (PIO)

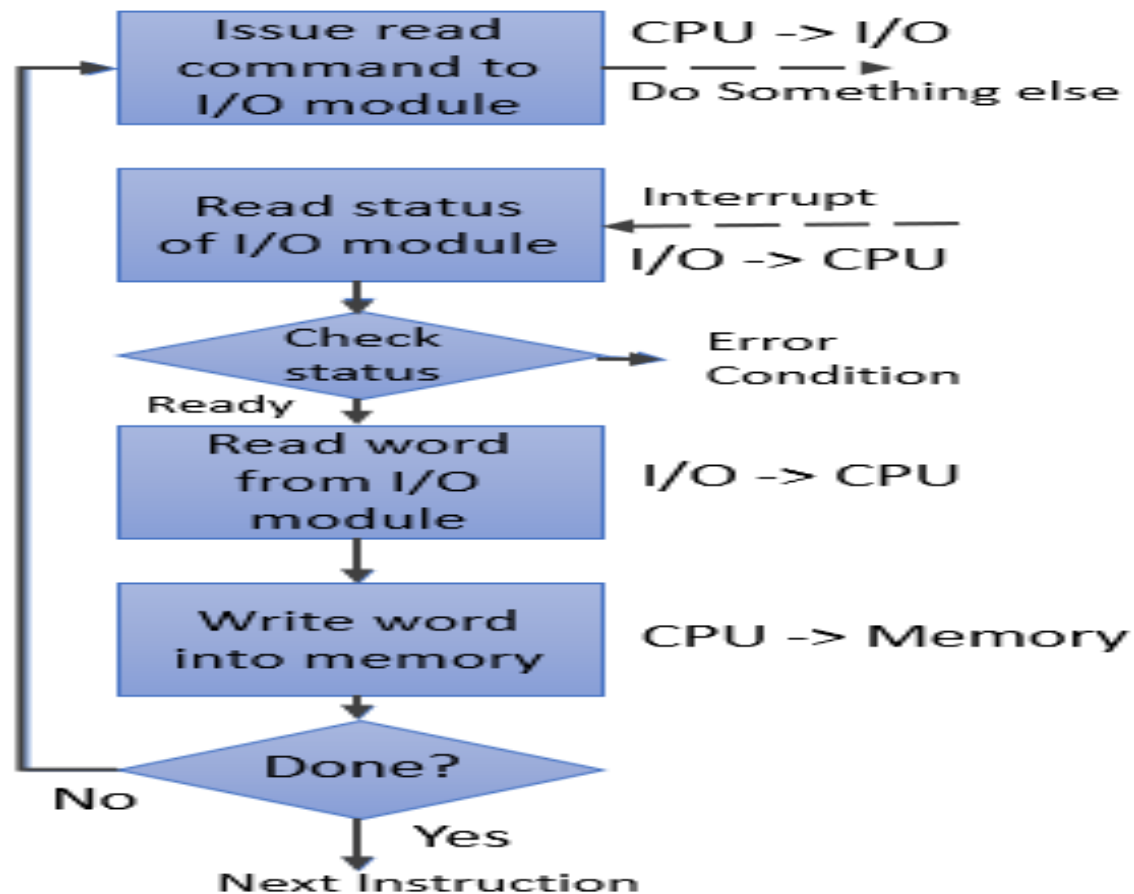
- Programmed I/O uses the I/O instructions written in the computer program.
- The instructions in the program initiate every data item transfer.
- Usually, the data transfer is from a memory and CPU register.
- This case requires constant monitoring by the peripheral device's CPU.
- Examples:
 - Reading data from a keyboard or a mouse where the CPU continuously polls the device for keypresses or mouse movements.
 - Writing data to a printer where the CPU initiates the print operation, checks the printer status, and transfers data in small chunks.



- Advantages:
- Programmed I/O is simple to implement.
- It requires very little hardware support.
- CPU checks status bits periodically.
- Disadvantages:
- The processor has to wait for a long time for the I/O module to be ready for either transmission or reception of data.
- The performance of the entire system is severely degraded.

Interrupt-Driven I/O

- In Previous section, we saw that the CPU is kept busy unnecessarily. We can avoid this situation by using an interrupt-driven method for data transfer.
- The interrupt facilities and special commands inform the interface for issuing an interrupt request signal as soon as the data is available from any device.
- In the meantime, the CPU can execute other programs, and the interface will keep monitoring the i/O device.
- Whenever it determines that the device is ready for transferring data interface initiates an interrupt request signal to the CPU.
- As soon as the CPU detects an external interrupt signal, it stops the program it was already executing, branches to the service program to process the I/O transfer, and returns to the program it was initially running.



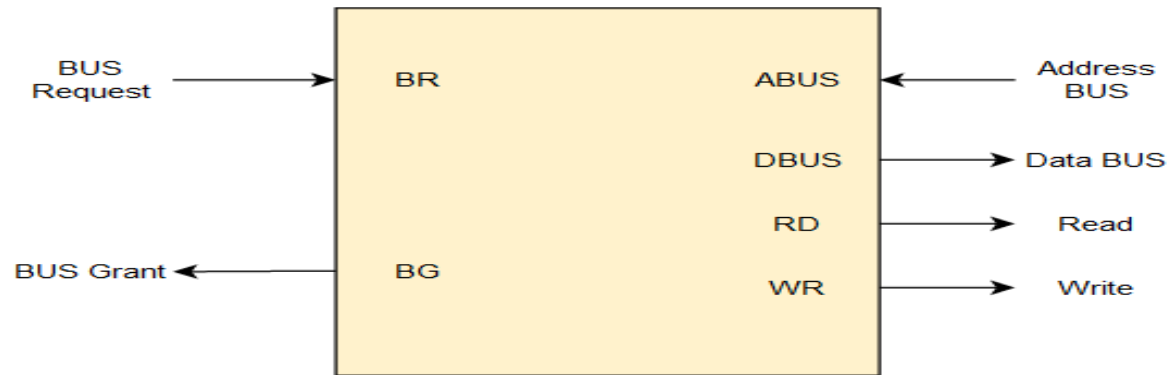
Interrupted I/O to Transfer Data from I/O Module to Memory

- Working of CPU in terms of interrupts:
- CPU issues read command.
- It starts executing other programs.
- Check for interruptions at the end of each instruction cycle.
- On interruptions:-
- Process interrupt by fetching data and storing it.
- See operation system notes.
- Starts working on the program it was executing.

- Advantages:
- It is faster and more efficient than Programmed I/O.
- It requires very little hardware support.
- CPU does not check status bits periodically.
- Disadvantages:
- It can be tricky to implement if using a low-level language.
- It can be tough to get various pieces of work well together.
- The hardware manufacturer / OS maker usually implements it, e.g., Microsoft.

Direct Memory Access (DMA)

- The data transfer between any fast storage media like a memory unit and a magnetic disk gets limited with the speed of the CPU.
- It will be best to allow the peripherals to directly communicate with the storage using the memory buses by removing the intervention of the CPU.
- This mode of transfer of data technique is known as Direct Memory Access (DMA).
- During Direct Memory Access, the CPU is idle and has no control over the memory buses.
- The DMA controller takes over the buses and directly manages data transfer between the memory unit and I/O devices.



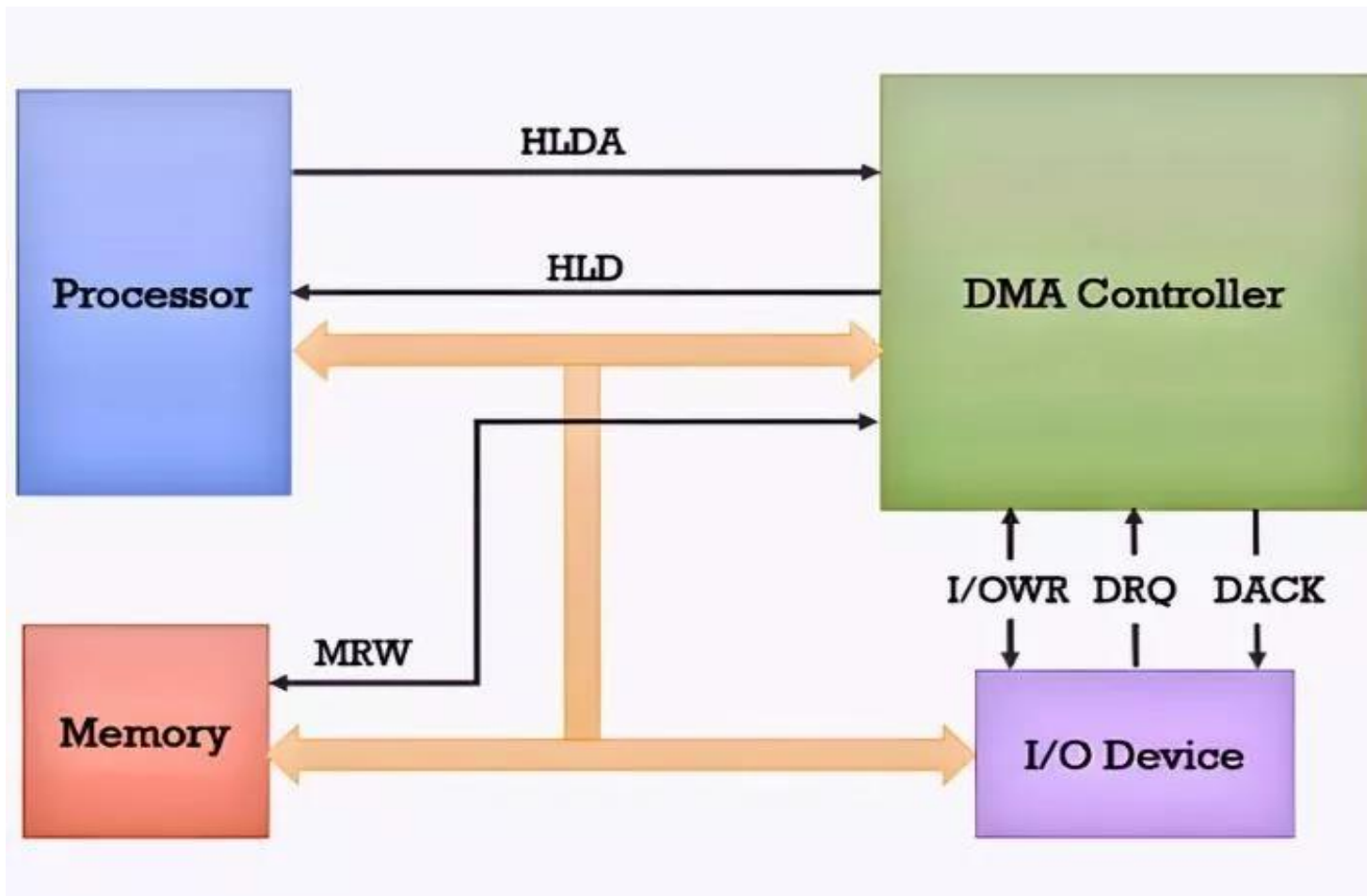
Bus Request - We use bus requests in the DMA controller to ask the CPU to relinquish the control buses.

Bus Grant - CPU activates bus grant to inform the DMA controller that DMA can take control of the control buses. Once the control is taken, it can transfer data in many ways.

Types of DMA transfer using DMA controller:

- Burst Transfer: In this transfer, DMA will return the bus control after the complete data transfer.
- A register is used as a byte count, which decrements for every byte transfer, and once it becomes zero, the DMA Controller will release the control bus.
- When the DMA Controller operates in burst mode, the CPU is halted for the duration of the data transfer.

- Cyclic Stealing: It is an alternative method for data transfer in which the DMA controller will transfer one word at a time.
- After that, it will return the control of the buses to the CPU.
- The CPU operation is only delayed for one memory cycle to allow the data transfer to “steal” one memory cycle.



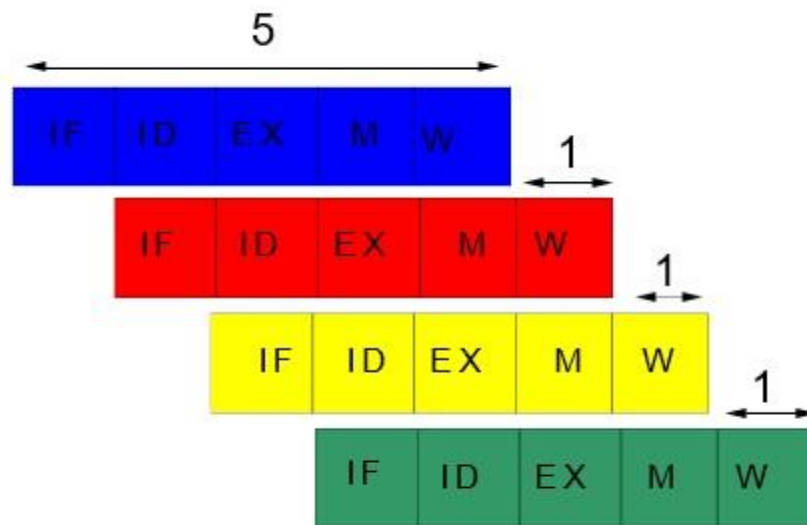
- Whenever an input-output device wants to transfer some data to the memory or from memory, it sends a DMA Request that can be referred to as DRQ to the DMA controller.
- The DMA controller accepts this (DMA Request)DRQ and asks the CPU to capture or hold a few clock cycles by requesting a Hold request(HLD).
- The CPU receives a hold request (HLD) from the DMA controller, releases the bus, and sends the hold acknowledgment (HLDA) to the DMA controller.

- After obtaining a hold acknowledgment (HLDA), the DMA controller approves the input-output device (DACK) to perform data transfer.
- The DMA controller takes power from the system bus and transfers data to or from memory.
- Once the data transfer completes, the DMA raises an interrupt to notify the processor about completing the data transfer function.
- The processor can control the bus again and begin processing where it left off.

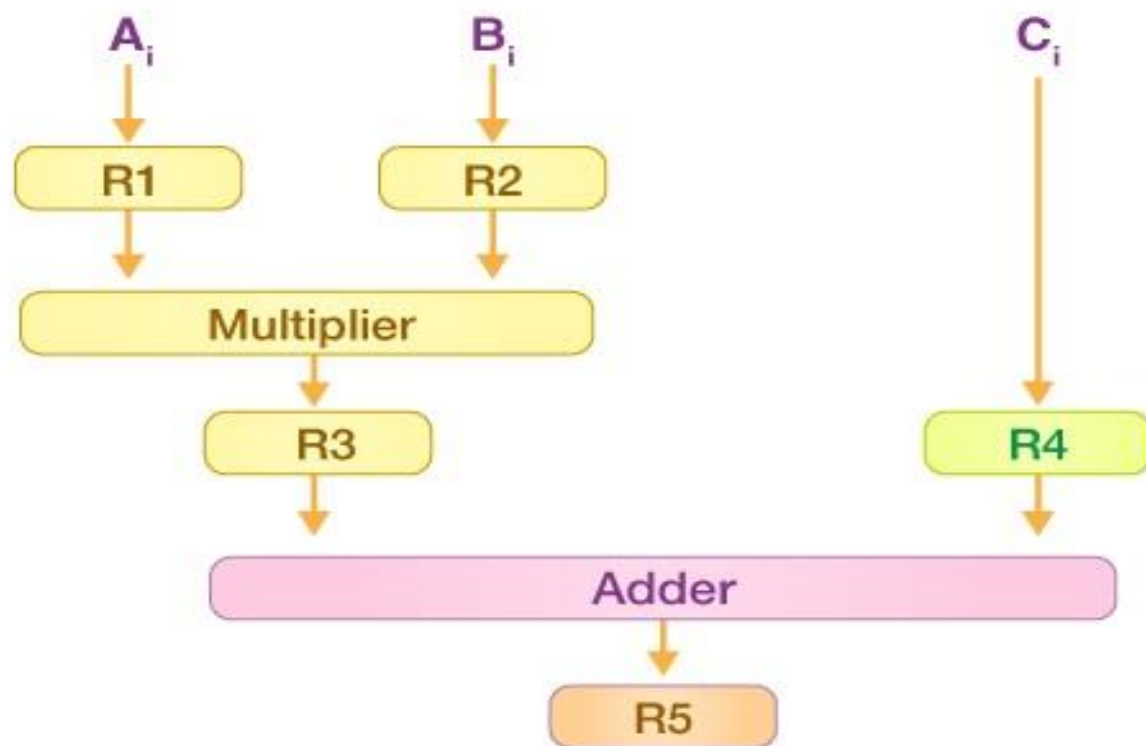
- Advantages
- It is faster in data transfer without the involvement of the CPU.
- It improves overall system performance and reduces CPU workload.
- It deals with large data transfers, such as multimedia and files.
- Disadvantages
- It is costly and complex hardware.
- It has limited control over the data transfer process.
- Risk of data conflicts between CPU and DMA.

Pipelining

- Pipelining is a technique for breaking down a sequential process into various sub-operations and executing each sub-operation in its own dedicated segment that runs in parallel with all other segments.
- The most significant feature of a pipeline technique is that it allows several computations to run in parallel in different parts at the same time.



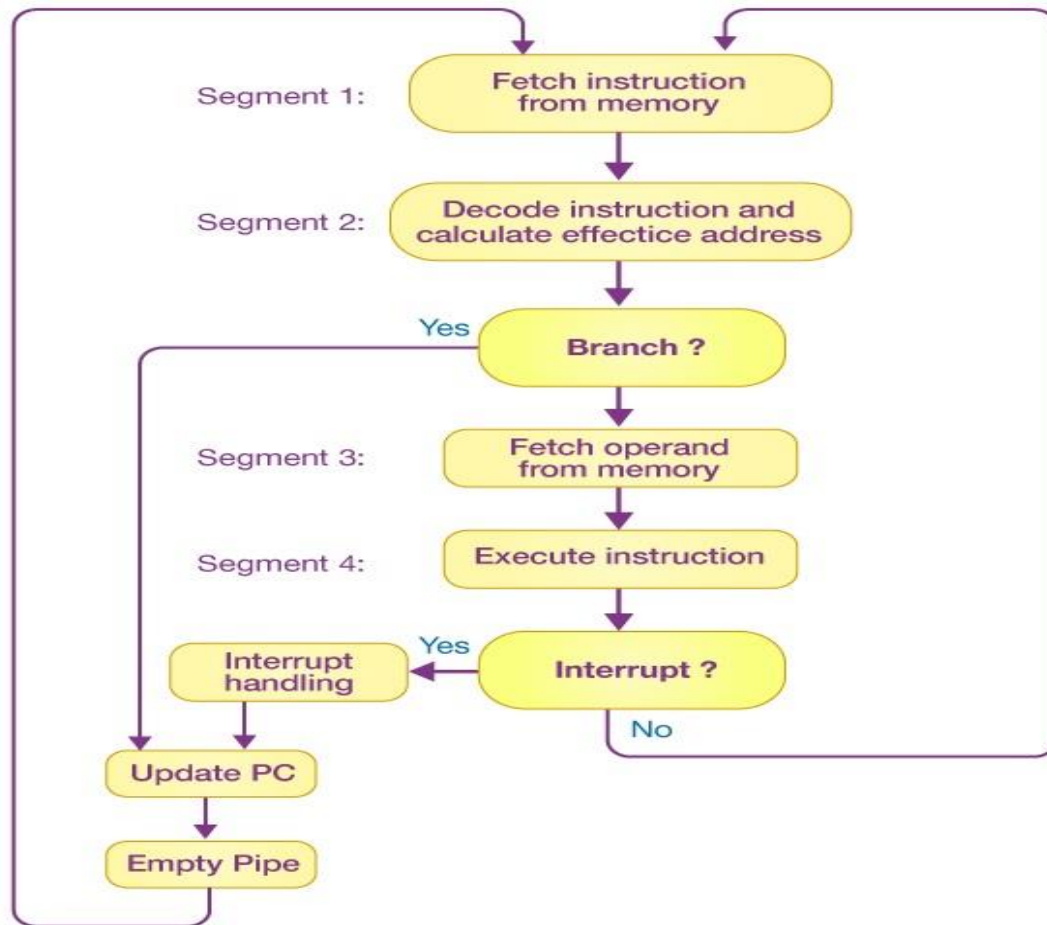
Four Pipelined Instructions



Instruction Pipeline

- Pipeline processing can happen not only in the data stream but also in the instruction stream. To perform tasks such as fetching, decoding and execution of instructions.
- Each step is carried out in its own segment, and various segments may take different amounts of time to process the incoming data.
- when multiple segments request memory access at the very same time, requiring one segment to wait unless and until the memory access of another is completed.

- Fetching the instruction from memory
- Decoding the obtained instruction
- Calculating the effective address
- Fetching the operands from the given memory
- Execution of the instruction
- Storing the result in a proper place



- Segment 1
- The implementation of the instruction fetch segment can be done using the FIFO or first-in, first-out buffer.
- Segment 2
- In the second segment, the memory instruction is decoded, and the effective address is then determined in a separate arithmetic circuit.

- Segment 3
- In the third segment, some operands would be fetched from memory.
- Segment 4
- The instructions would finally be executed in the very last segment of a pipeline organisation.

Pipeline Hazards

- CPU's speed is limited by memory. There's one more case to consider, i.e. a few instructions are at some stage of execution in a pipelined design.
- There is a chance that these sets of instructions will become dependent on one another, reducing the pipeline's pace.
- Dependencies and hazards since they are used interchangeably in computer architecture.
- A hazard, in essence, prevents an instruction present in the pipe from being performed during the specified clock cycle.

Types of Pipeline Hazards

- The three different types of hazards in computer architecture are:
 - 1. Structural
 - 2. Data
 - 3. Control

Structural Hazard

- Hardware resource conflicts among the instructions in the pipeline cause structural hazards.
- Memory, a GPR Register, or an ALU might all be used as resources here.
- When more than one instruction in the pipe requires access to the very same resource in the same clock cycle, a resource conflict is said to arise.
- In an overlapping pipelined execution, this is a circumstance where the hardware cannot handle all potential combinations

Data Hazards

- Data hazards in pipelining emerge when the execution of one instruction is dependent on the results of another instruction that is still being processed in the pipeline.
- The order of the READ or WRITE operations on the register is used to classify data threats into three groups.

Control Hazards

- Branch hazards are caused by branch instructions and are known as control hazards in computer architecture.
- The flow of program/instruction execution is controlled by branch instructions.
- Remember that conditional statements are used in higher-level languages for iterative loops and condition testing (correlate with while, for, and if case statements).
- These are converted into one of the BRANCH instruction variations.
- when the decision to execute one instruction is reliant on the result of another instruction, such as a conditional branch, which examines the condition's consequent value, a conditional hazard develops.

Super Scaler Processing

- A type of microprocessor that is used to implement a type of parallelism known as instruction-level parallelism in a single processor to execute more than one instruction during a CLK cycle by dispatching simultaneously various instructions to special execution units on the processor.
- A **scalar processor** executes single instruction for each clock cycle
- A **superscalar processor** can execute more than one instruction during a clock cycle.

- a superscalar processor is a CPU that executes above one instruction for each CLK cycle because processing speeds are simply measured in CLK cycles for each second. Compared to a scalar processor, this processor is very faster.

Features

- Superscalar architecture is a parallel computing technique utilized in various processors.
- In a superscalar computer, the CPU manages several instruction pipelines to perform numerous instructions simultaneously during a clock cycle.
- Superscalar architectures include all pipelining features although there are several instructions executing simultaneously within the same pipeline.

	1	2	3	4	5 	
						Clock Periods
I1	F	D	E	W		
I2	F	D	E	W		
I3		F	D	E	W	
I4		F	D	E	W	
Instructions						

- The superscalar processor pipeline architecture includes two pipelines and four stages fetch, decode, execute & result write back.
- It is a 2-issue superscalar processor which means at a time two instructions will fetch, decode, execute and result write back.
- The two instructions I1 & I2 will at a time fetch, decode, execute and write back in every clock period.
- Simultaneously in the next clock period, the remaining two instructions I3 & I4 will at a time fetch, decode, execute and write back. So, in five clock periods, it will execute 4 instructions in a single pipeline.

Processor-level parallelism

- Processor-level parallelism in computer architecture refers to the capability of a system to execute multiple instructions or tasks concurrently on multiple processors or cores, thus improving overall performance.
- This is achieved through various techniques like instruction pipelining, multiple cores within a single processor (multicore architecture), and parallel processing using multiple processors.

Types of Processor-Level Parallelism

- **Instruction-Level Parallelism (ILP):**
- This involves executing multiple instructions from the same instruction stream simultaneously within a single processor. Techniques like pipelining and superscalar architectures enhance ILP.

- **Thread-Level Parallelism (TLP):**
- This exploits parallelism by executing multiple threads concurrently, often using multiple cores within a single processor or multiple processors. Techniques like simultaneous multithreading (SMT) and multi-core processors enable TLP.

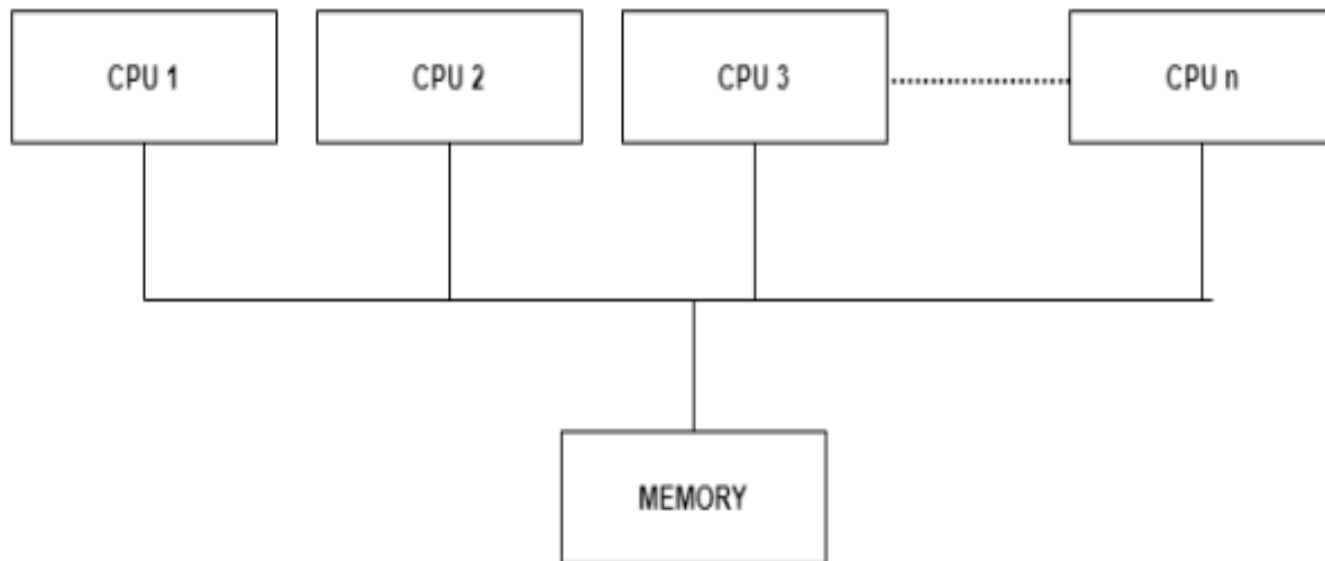
- **Data-Level Parallelism:**
- This focuses on processing multiple data elements simultaneously using specialized hardware or software. Examples include SIMD (Single Instruction, Multiple Data) and array processors.

Examples of Processor-Level Parallelism

- **Multi-core Processors:**
- Modern desktop and server processors typically include multiple cores, enabling the execution of multiple threads or tasks concurrently.
- **GPUs (Graphics Processing Units):**
- GPUs are designed for high levels of parallelism and are well-suited for tasks like image rendering, scientific simulations, and machine learning.

Multiprocessor

- A multiprocessor is a computer system with two or more central processing units (CPUs), with each one sharing the common main memory as well as the peripherals.
- This helps in simultaneous processing of programs.
- The key objective of using a multiprocessor is to boost the system's execution speed, with other objectives being fault tolerance and application matching..



Multiprocessing Architecture

- The term "multiprocessor" can be confused with the term "multiprocessing". While multiprocessing is a type of processing in which two or more processors work together to execute multiple programs simultaneously, multiprocessor refers to a hardware architecture that allows multiprocessing.
- A multiprocessor is regarded as a means to improve computing speeds, performance and cost-effectiveness, as well as to provide enhanced availability and reliability

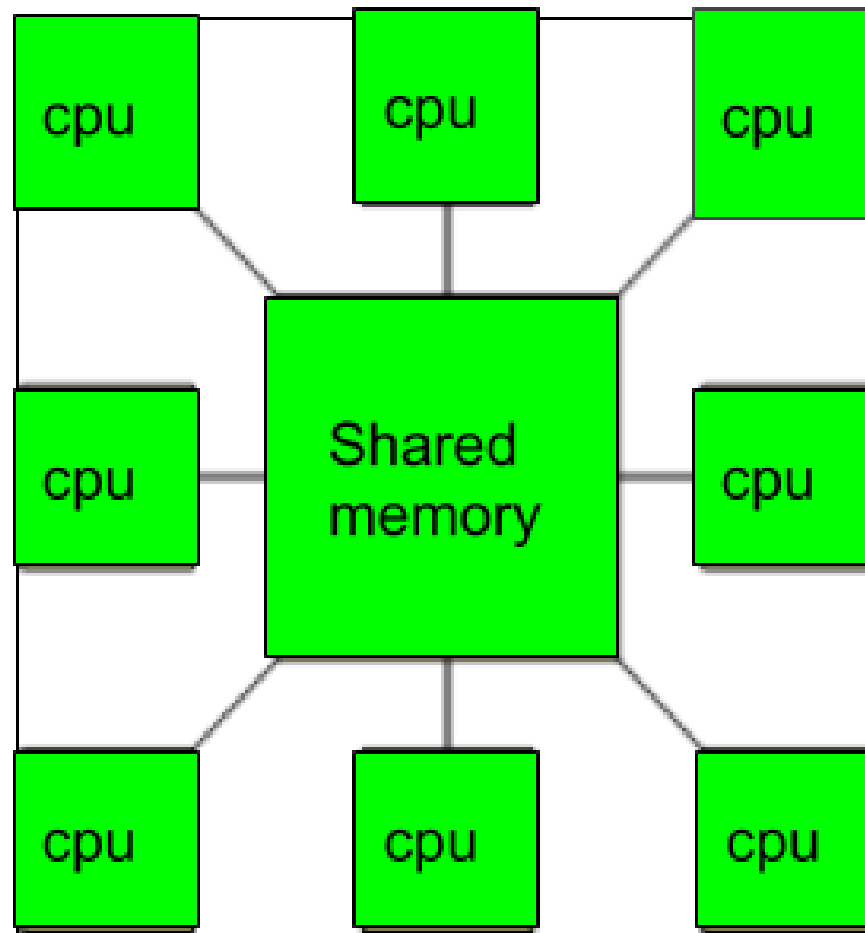
Types of multiprocessor systems:

Symmetric Multiprocessors

- In these types of systems, each processor contains a similar copy of the operating system and they all communicate with each other. All the processors are in a peer to peer relationship i.e. no master - slave relationship exists between them.
- An example of the symmetric multiprocessing system is the Encore version of Unix for the Multimax Computer.

Asymmetric Multiprocessors

- In asymmetric systems, each processor is given a predefined task. There is a master processor that gives instruction to all the other processors. Asymmetric multiprocessor system contains a master slave relationship.
- Asymmetric multiprocessor was the only type of multiprocessor available before symmetric multiprocessors were created. Now also, this is the cheaper option.



- The interconnection among two or more processor and shared memory is done with three methods
- Time shared common bus
- Multiport memories
- Crossbar switch network